
VLM-Verified Counterfactual Hindsight for Sparse-Reward Manipulation*

Daniel Grant Parshawn Gerafian Matei Gardea
Department of Electrical Engineering and Computer Sciences
University of California, Berkeley
{dgrant, pgerafian, mgardea}@berkeley.edu

Extended Abstract

Problem. Sparse-reward goal-conditioned manipulation is a temporal credit-assignment problem: in Gymnasium-Robotics Fetch (Push / PickAndPlace / Slide), a randomly-initialized policy succeeds in $< 1\%$ of episodes, and the lone terminal reward bit at the end of a 50-step trajectory decays exponentially through TD bootstrapping. Hindsight Experience Replay (HER) relabels failed trajectories with achieved goals but provides no signal about *which* timestep caused failure; standard PER cannot recover this in sparse regimes because the TD error is itself the bottleneck. Concurrent VLM-RB [27] uses frozen VLMs as an *additive* priority bias; we argue the *multiplicative* form is the principled choice and pair it with a simulator-verified counterfactual channel.

Methodological contributions. (1) **Semantic PER as importance-sampled posterior reweighting:** we recast VLM-guided replay as a trajectory-conditional posterior $q_\phi(t^* | \tau)$ over the failure-causing timestep, and show that multiplying the standard TD-error priority by q_ϕ admits a clean IS-correction analysis with an explicit bias bound (Eq. 10): VLM miscalibration inflates sampling *variance* but never corrupts the value target; the additive VLM-RB mixture implicitly retargets the Bellman loss. (2) **VLM-Verified Counterfactual Hindsight:** we close the dominant failure mode of language-only counterfactuals (100% teleport-collapse of Opus 4.7 on FetchPush) by asking the VLM for a *corrective action sequence* rather than a hindsight goal, executing it in a simulator fork, and admitting only physics-consistent, reward-positive relabels into the buffer (confidence 1.0, zero modeling error).

Headline findings. (i) **Verified-CF matches VLM-CF in aggregate, wins on Slide.** At 500k SAC+HER+verified-CF updates the verified-CF agent reaches mean success 0.606 across the three Fetch envs (0.85/0.35/0.617 on Push / PickAndPlace / Slide), tying VLM-CF (0.622 mean, $\Delta = -0.016$) and *exceeding* it on FetchSlide (0.617 vs. 0.55; +0.45 over PER@3M, +0.43 over HER@250k); Fig. 6. (ii) **Pre-registered kill verdict, honored.** The Phase-1 Oracle-CF study violated the +0.10 kill threshold on FetchPickAndPlace at the 250k decision horizon ($\Delta = -0.05$); we honored the kill and pivoted the headline to the IS-posterior framing and the verifier. A post-hoc 1M re-run reaches mean ≈ 0.40 on PickAndPlace (vs. HER@250k of 0.167); we report it as transparency, not retraction. (iii) **Prompt-architectural failure.** A two-vendor three-model sweep (GPT-4o / Opus 4.7 / Sonnet 4.5) on 6 synthetic and 10 real Fetch failures locates teleport-collapse as prompt-architectural (0/6 vs. 4/4 for (GPT-4o, `achieved_goal`) vs. Opus), not model-specific. (iv) **Cross-task transfer.** A single prompt template yields 12/12 parse / task-relevance / plausibility on Push / PickAndPlace / Slide.

Honest negatives and roadmap. The verified-CF channel exhibits a cold-start regime (Fetch-PickAndPlace seed 42: 100% verifier-rejection over the first ~ 80 VLM calls) in which it reduces to a strictly costlier HER; the two failure modes are asymmetric (Semantic-PER variance inflation is

*CS 285 Final Project, Spring 2026. UC Berkeley, Department of EECS.

bounded; verified-CF write-throughput collapse is not). §3 derives the IS-posterior framing and bias bound; §4 presents Semantic PER, the prompt-design analysis, and the verifier; §5 reports the head-line, prompt, and kill results; §6 discusses limitations. Appendices contain prompts, hyperparameters, and the full bias-bound proof.

1 Introduction

The core difficulty of sparse-reward manipulation is temporal credit assignment: with only one reward bit at the end of a 50-step episode, every standard TD-error signal decays exponentially back from the terminal timestep, leaving the bulk of each trajectory effectively invisible to the optimizer. In Gymnasium-Robotics Fetch environments—a canonical benchmark family with a 5 cm goal-achieved threshold and continuous end-effector control—a randomly-initialized policy will succeed in under 1% of episodes, so the early replay buffer contains almost no positive signal to propagate forward. Standard prioritized experience replay (PER) [26] sharpens the sampling distribution toward high-TD-error transitions, but this sharpening is itself downstream of a credit signal that decays exponentially away from the rare goal-achievement timestep. Hindsight Experience Replay (HER) [1] partially repairs this by relabeling failures as successes for an alternative achieved goal, converting every failed trajectory into at least one positive TD signal—but the relabel target is constrained to the agent’s actually-realized trajectory, so the most instructive transition (the one where the failure became inevitable) receives no special treatment.

Recent work has explored using foundation Vision-Language Models (VLMs) as “oracles” inside the RL loop, both for reward shaping [24, 13, 29] and, very recently, for replay-buffer prioritization [27]. We argue these uses are best framed not as engineering recipes but as *importance-sampled stochastic optimization with a foundation-model proposal distribution*: the VLM is a frozen approximation to the (intractable) posterior $p(t^* | \tau)$ over which transitions caused an episode’s outcome.

This paper makes four contributions.

(1) An IS-posterior framing of VLM-guided replay (Section 3). We re-derive prioritized DQN [26], retrace [17], V-trace [6], and Sharony et al.’s VLM-RB [27] inside a non-uniform-replay taxonomy, locating our scheme as *proposal-shaping* with a frozen foundation-model credit-assignment oracle—the vision-modality, prioritized-replay instantiation of Pignatelli et al. [21]’s exogenous-FM-credit-oracle program in robot RL. We choose the multiplicative form because it admits a clean IS-correction analysis (the standard PER weight remains trajectory-conditionally well-defined); the additive mixture used by VLM-RB [27] implicitly retargets the loss to a λ -weighted objective.

(2) Semantic PER with a failure-direction signal (Section 4.1). Sharony et al. score 32-frame clips for goal-satisfaction; we ask the VLM the strictly more informative question—*which single timestep made this trajectory fail?*—and multiply the standard PER priority by a bounded boost over a window around that timestep. The scheme degenerates gracefully to PER when the VLM is uncommitted ($w_{\max} = 1$), adds no VLM-specific schedule on top of PER’s standard β anneal (in contrast to VLM-RB’s $\lambda_t : 0 \rightarrow 0.5$ mixture warm-up), and supports a strictly-corrected IS estimator (Appendix C.1, Eq. (9)); our running implementation uses PER’s IS weight as a controlled under-correction (V-trace-analogous), deferring the strictly-correct ablation to future work. Crucially, unlike VLM-as-reward approaches [24, 33, 19] that splice the VLM signal into the TD target, our scheme leaves the target untouched: the env’s true sparse reward is recomputed on every sampled transition. VLM miscalibration shifts the sampling distribution, not the value target; a mis-calibrated VLM increases variance rather than introducing Q-bias.

(3) VLM-Verified Counterfactual Hindsight (Section 4.3). A direct prompt for an alternative *achieved goal* elicits a teleport-collapse failure mode in 100% of Claude Opus 4.7 calls on FetchPush: the VLM returns `corrective_position = desired_goal`, a degenerate zero-information output. We close this loophole *by construction* by asking instead for a corrective *action sequence* and executing it in a fork of the training simulator from the failure-timestep state. A four-vector action cannot teleport a 50 g block by 50 cm under MuJoCo dynamics; the failure mode is structurally inexpressible. The mechanism instantiates the *generator-verifier* paradigm [35] in robot RL—the VLM generates, the simulator verifies—and only trajectories whose sparse reward fires under the unmodified environment dynamics enter the buffer, with confidence 1.0 and zero modeling error.

(4) Empirical analysis: prompt design, VLM-family robustness, and cross-task transfer (Sections 5.3–5.5). A head-to-head GPT-4o vs. Claude Opus 4.7 sweep on six synthetic Fetch failures identifies a strictly-dominating configuration: (GPT-4o, `achieved_goal`) eliminates teleport-collapse (0/6 vs. Opus’s 4/4), ties for highest plausibility (0.79), and trails only Opus’s degenerate goal-copying on goal-progress (0.83 vs. 0.97). A second prompt sweep on ten *real* SAC-failure episodes confirms the prompt-architectural fix transfers to a third VLM family (Sonnet 4.5: 75% $\rightarrow$ 0% teleport on PickAndPlace). A 12-episode cross-task pilot achieves 12/12 task-relevant annotations on Push/PickAndPlace/Slide with a single prompt template, evidence that a foundation-VLM credit-assignment oracle generalizes across structurally-different manipulation tasks where any learned priority head would require per-task fitting.

The remainder of the paper is organized as follows. Section 2 surveys HER-family relabeling, the PER lineage, and the recent use of foundation models in RL, explicitly situating our two mechanisms in the relabel-verification and proposal-shaping subfamilies. Section 3 derives the IS-posterior framing and the off-policy-correction taxonomy that positions Semantic PER and VLM-RB. Section 4 presents the implementation of Semantic PER (§4.1), the prompt-design analysis and teleport-collapse formalization (§4.2), and the VLM-Verified Counterfactual Hindsight mechanism (§4.3). Section 5 reports SAC+HER baselines on the three Fetch environments, the head-to-head prompt-design sweep, cross-task transfer evidence, real-data validation on 10 agent-generated failures, and the current status of two in-flight experiments—a HER vs. Oracle-CF kill test and the full VLM-CF sweep on Modal. Section 6 discusses a pipeline-bug discovery that re-frames our headline GPT-4o curve, small- n caveats, and eight forward-looking limitations pre-registering the camera-ready experimental agenda.

2 Related Work

HER and goal-conditioned RL. Hindsight Experience Replay [1] is the dominant solution to sparse-reward goal-conditioned manipulation; the future relabel strategy with $k = 4$ remains a strong baseline a decade later. HER’s core move is to convert a failed trajectory τ into a positive demonstration for some achieved goal $g' \in \{a_{g_t}\}_{t=0}^T$ on the trajectory itself, exploiting the fact that the sparse reward function $r(s, a, g) = -\mathbb{I}[\|a_g - g\| > d_{th}]$ admits cheap re-evaluation under a swapped goal. We organize the recent HER-family literature along three orthogonal extensions of this move—the *relabel target*, the *relabel proposal*, and the *relabel verification step*—and place our work in the third. **Relabel-target extensions** change *which* alternative goals are considered. Single-step Next-Future [18] restricts relabel candidates to ag_{t+1} for sample-efficiency. Contact-Energy Hindsight Prioritization [25] uses privileged end-effector/object state to upweight transitions whose contact-energy correlates with goal achievement—a state-geometry analogue of our heuristic Oracle v3 (§4.1). **Relabel-proposal extensions** change *how* alternative goals or trajectories are synthesized rather than read off the agent’s own data. Causal Action Influence Counterfactual Augmentation [28] generates additional transitions on Fetch and Franka-Kitchen by counterfactually intervening on object positions under a learned causal-influence proxy. The HER-for-LM-agents line [10, 4] re-writes failed trajectories as language-instruction relabels, with AgentHER explicitly adding multi-judge VLM verification to reduce relabel noise (5.9% $\rightarrow$ 2.3%), an early acknowledgment that language-only counterfactuals require an external acceptance test. **Relabel-verification extensions** add an explicit gate between the proposed relabel and the buffer. HInt/NCII [3] asks “would the target object’s dynamics change if the cause object were removed?” using a *learned* dynamics model as the verifier; GCHR [14] regularizes the relabel proposal via a hindsight-conditioned policy term. Our verified-counterfactual mechanism (§4.3) sits in this third family but verifies in the *exact training simulator* (zero modeling error) and verifies an *action sequence* rather than a hindsight goal—a stricter test because it requires physics-consistent execution under the original dynamics, not merely a dynamics-model consistency check.

Prioritized replay and off-policy correction. Prioritized DQN [26] samples in proportion to $(|\delta| + \epsilon)^\alpha$ and reweights every sampled gradient by the annealed importance-sampling correction $w_{IS} = (|\mathcal{D}| \cdot \mu_P)^{-\beta}$, $\beta: \beta_0 \rightarrow 1$. The IS correction is the load-bearing structural property of PER: in the $\beta = 1$ limit the reweighted gradient is an unbiased estimator of the uniform-replay loss $\mathbb{E}_U[\ell]$ under self-normalized importance sampling, with the proposal μ_P supplying variance reduction [26, Sec. 3.4]. Multi-step off-policy correction generalizes this idea: Retrace [17] clips per-step IS ratios to bound the trace’s variance under truncation-induced bias; V-trace [6] doubly-clips the target and behavior ratios in a distributed actor-learner setup. In each case the proposal is a fixed behavior or

policy distribution; the correction inherits its form from the proposal–target ratio. The PER lineage in 2024–2026 has explored *auxiliary* proposal signals beyond raw TD-error while preserving the multiplicative “ $\mu_P \cdot g$ ” structure: ReaPER multiplies the PER priority by a per-transition reliability estimate [23]; RPE-PER replaces $|\delta|^\alpha$ with a reward-prediction-error proposal [34]; Prioritized Generative Replay multiplies the priority by a generative-model relevance score (curiosity or value) on a learned synthetic-experience stream [30]; the non-uniform-memory line shows that almost any non-uniform proposal beats uniform under modest conditions [12]. Ma et al. [15] address a different failure mode—priority staleness when the policy itself is a rapidly-evolving LLM/VLM—via age-decay weights. Within this taxonomy our Semantic PER (§4.1) is a *proposal-shaping* scheme whose auxiliary signal w_{sem} (i) is *exogenous* (a frozen foundation-VLM posterior, not a critic-internal estimate), (ii) is *trajectory-level* (one VLM call per failed episode, then a kernel-smeared weight applied to every slot in that episode), and (iii) combines *multiplicatively* with μ_P , which is the family choice that preserves PER’s IS-correction interpretation as derived in §3. The contrasting design choice in the very recent VLM-RB [27] is an *additive* mixture $\lambda\mu_{\text{VLM}} + (1 - \lambda)\mu_U$; we return to this contrast in Table 2 and in Section 4.1.

Foundation models in RL. The 2024–2026 literature on foundation models in RL is best understood through *where in the training loop* the VLM/LLM output is consumed. **TD-target integration** couples the VLM output directly to the Q-regression target: per-timestep reward models [33, 19], affordance-shaped potential functions [13], and pairwise-preference reward models [24, 29, 31] all inject VLM calibration error into Q , propagating it into every downstream value estimate. Duan et al. [5] fine-tune a VLM (AHA) on synthetic robotic failures and downstream-refine LLM-generated reward code (Eureka pipeline); AHA integrates via the TD target too. **Data-stream integration** instead modifies *which* transitions enter the buffer or are prioritized for sampling—leaving the TD target untouched. CAST [8] and ECHO [10] relabel failed trajectories; CoSo [7] re-weights per-token counterfactual credits; AgentHER [4] adds a VLM verification gate on relabeled LM-agent episodes. Promptable feature encoders [2] operate earlier still, in the perception stack.

Our system is data-stream integration: we use the frozen VLM output *only* to reshape the replay-sampling distribution μ ; the TD target retains the environment’s true sparse reward. VLM miscalibration therefore appears only as sampling-distribution variance—correctable by the IS weight (Section 3)—not as Q-target bias. This is the clean prediction P2 of Section 3 makes explicit: *semantic-PER tracks standard PER when the VLM is poorly calibrated and pulls ahead when it is reliable, without diverging*. TD-target methods cannot make this prediction.

We differ from AHA along three orthogonal axes beyond integration point: (i) AHA *describes* failures; we *localize* to a single timestep as a credit-assignment signal. (ii) AHA proposes no corrective actions; we do, and gate them on simulator verification. (iii) AHA uses a fine-tuned VLM on a supervised failure corpus; we query a frozen general-purpose VLM zero-shot, requiring no failure annotations. An AHA-fine-tuned VLM could plausibly replace zero-shot GPT-4o inside our protocol as a drop-in module.

Foundation-model credit assignment is an emerging sub-thread: Khandoga et al. [11] mask reasoning spans in an LLM policy to identify causally-influential tokens, in the lineage of Mesnard et al. [16] and the credit-assignment survey by Pignatelli et al. [20]. Pignatelli et al. [21] establish the foundation-model-as-credit-oracle program by showing that an LLM can serve as a zero-shot credit oracle via subgoal decomposition in text-based environments. We place Semantic PER as the *first vision-modality, prioritized-replay instantiation* of that program: the oracle is a frozen VLM (not an LLM), the domain is goal-conditioned robot manipulation (not text adventures), and the oracle output is consumed by the replay-sampling distribution rather than by a planner, with the multiplicative-with-PER coupling and the simulator-verified counterfactual gate (§4.3) as the two methodological additions on top of CaLM’s planner-only consumption. This places our contribution inside the emerging causal-credit-assignment family rather than the reward-shaping family—a more useful placement than “VLM in replay” (an engineering label) and a more accurate one than “a new credit-oracle category” (CaLM precedes us).

Generator–verifier and verified counterfactual reasoning. Our verification protocol (Section 4.3) sits inside the *generator–verifier* paradigm now popular in LLM reasoning: the generator emits candidates, a verifier (learned or symbolic) accepts the correct ones [35]. We instantiate this in robotics with a foundation-VLM generator and a *symbolic* verifier—the unmodified training simulator—whose

Table 1: VLM-Guided Experience Replay (Sharony et al. 2026) vs. this paper.

Axis	VLM-RB [27]	This paper
Signal direction	Goal satisfaction (success)	Failure-causing transitions
Granularity	32-frame sub-trajectory clip	Single timestep $\pm$ window
Blending	Mixture-with-uniform; $\lambda_t : 0 \rightarrow 0.5$ warm-up	Multiplicative; degenerates to PER
Headroom	—	Heuristic privileged-state oracle
CF relabel	—	VLM-proposed action + sim-fork verification
Cross-task	Per-benchmark tuning of λ , TD multiplier	One prompt template across 3 Fetch envs
Benchmarks	MiniGrid, OGBench (DQN/IQN/SAC/TD3)	Gym-Robotics Fetch (SAC)

decisions are correct by construction. This instantiation has a structural advantage beyond zero modeling error: the verifier is *identical* to the environment on which the policy trains, so every accepted relabeled transition is not merely *physically plausible* but *in-distribution for the training MDP*—a guarantee that learned world-model verifiers [32] cannot provide when their training distribution differs from the RL task’s dynamics.

The most related prior work in robotics is the line on counterfactual data augmentation for goal-conditioned RL [28, 3, 8], which differs from our approach along the verifier axis: CAIAC [28] uses a *causal-influence proxy* (estimated interventional effect on object trajectory); HInt/NCII [3] uses a *learned dynamics model* (modeling error bounded but nonzero); CAST [8] uses a *frozen VLM plausibility score* as the acceptance signal (no guarantee of physics consistency). Each of these three choices produces a verifier with at least one of: non-zero modeling error, domain-shift sensitivity, or an implicit distributional assumption about the RL task. We use the training simulator directly, which eliminates all three failure modes at the cost of additional sim-fork compute per counterfactual proposal.

Differentiation from VLM-RB. The concurrent (Feb 2026) VLM-RB [27] is the closest published work: a frozen VLM scores sub-trajectory clips for prioritized replay on MiniGrid and OGBench, reporting 11–52% higher success and 19–45% better sample efficiency. The two systems differ along five orthogonal axes summarized in Table 1, with two methodological cores worth emphasizing. **First**, VLM-RB asks “is this sub-trajectory good?” (success-direction scoring) and mixes the answer *additively* with uniform sampling $\mu = \lambda\mu_{\text{VLM}} + (1 - \lambda)\mu_U$; we ask the strictly more informative “which single timestep was outcome-causing?” and combine the answer *multiplicatively* with TD-error $\mu \propto \mu_P \cdot w_{\text{sem}}$. The multiplicative form is what preserves PER’s importance-sampling-correction interpretation (Section 3); the additive mixture implicitly retargets the loss to a λ -weighted objective that VLM-RB does not correct for. **Second**, we ship a verified-counterfactual-hindsight mechanism (Section 4.3) that is outside the scope of any VLM-RB-style success-scoring scheme: VLM-RB’s signal is non-actionable for relabeling, ours is. To our knowledge this paper is the first to (a) use a VLM-localized failure timestep as a credit-assignment signal and (b) verify counterfactual relabels in the same simulator on which the policy trains.

3 Theoretical Motivation: VLM-Guided PER as Posterior Reweighting

Setup. Let $\tau = (s_0, a_0, r_0, \dots, s_T)$ be an episode and $i \in \{0, \dots, T-1\}$ a transition index. Off-policy value learning minimizes a replay-sampled regression

$$\mathcal{L}(\theta) = \mathbb{E}_{i \sim \mu}[\ell(\delta_i(\theta))], \quad \delta_i = r_i + \gamma Q(s_{i+1}, \pi(s_{i+1})) - Q(s_i, a_i), \quad (1)$$

over a buffer $\mathcal{D}_B$. Standard PER [26] uses $\mu_P(i) \propto (|\delta_i| + \epsilon)^\alpha$ and reweights each gradient by the annealed IS correction

$$w_{\text{IS}}(i) = (|\mathcal{D}_B| \mu_P(i))^{-\beta}, \quad \beta: \beta_0 \rightarrow 1, \quad (2)$$

recovering an unbiased estimator of the uniform loss in the limit.

Table 2: Non-uniform replay and off-policy correction. Retrace / V-trace correct policy mismatch in multi-step returns; PER / VLM-RB / Sem. PER are replay-distribution schemes over a fixed buffer. Semantic PER is the only scheme whose modifier is exogenous and trajectory-level.

Method	Proposal μ	Correction	Bias source
Retrace [17]	Behavior μ_b	Clipped IS c_s	IS-ratio truncation
V-trace [6]	Behavior μ_b	Doubly-clipped $\bar{\rho}, \bar{c}$	Both-ratio truncation
PER [26]	$ \delta ^\alpha$	$(\mathcal{D} \mu_P)^{-\beta}$	Anneal $\beta < 1$
VLM-RB [27]	$\lambda\mu_{\text{VLM}} + (1-\lambda)\mu_U$	none	Mixture obj. $\neq$ PER
Sem. PER (ours)	$\mu_P \cdot w_{\text{sem}}$ (§3)	$w_{\text{IS},P}$ (under-corr.)	$q_\phi + \text{under-corr.}$

The VLM as a posterior. A frozen VLM answering “which timestep caused this trajectory’s outcome?” approximates the posterior $p(t^* | \tau)$ over the latent outcome-causing timestep; write $q_\phi(t^* | \tau)$ for the approximation. We define a per-transition semantic boost as the expected kernel weight under the posterior:

$$w_{\text{sem}}(i; \tau) := \mathbb{E}_{t^* \sim q_\phi(\cdot | \tau)} [K_W(i; t^*)], \quad K_W(i; t^*) = 1 + (w_{\text{max}} - 1) \mathbb{1}[|i - t^*| \leq W], \quad (3)$$

with window half-width W and bounded boost $w_{\text{max}} \geq 1$. The **Semantic PER proposal** is the multiplicative product

$$\boxed{\mu_{\text{Sem}}(i) \propto \mu_P(i) \cdot w_{\text{sem}}(i; \tau_i)} \quad (4)$$

μ_P captures *learning progress*; w_{sem} captures *causal influence*. When q_ϕ is near-uniform we degenerate to PER. Our implementation uses hard elicitation (a chain-of-thought returns a single $\hat{t} = \arg\max_{t^*} q_\phi$, equivalent to a Dirac at $\hat{t}$); the bias bound (Eq. 10, Appendix C.1) controls this collapse via the $w_{\text{max}} - 1$ multiplier. A soft-elicitation ablation is pre-registered.

Why multiplicative? Two reasons. (P1) IS-correction compatibility. Under $\mu_{\text{Sem}} \propto \mu_P \cdot w_{\text{sem}}$ the trajectory-conditional w_{sem} factor cancels in the normalized $\mathbb{E}_{\mu_{\text{Sem}}} [w_{\text{IS},P} \ell]$, so using the PER weight $w_{\text{IS},P} = (|\mathcal{D}_B| \mu_P)^{-\beta}$ as an under-correction yields a bounded V-trace-analogous bias (Appendix C.2). An additive mixture $\lambda\mu_q + (1-\lambda)\mu_P$ [27] instead shifts the $\beta \rightarrow 1$ limit to a λ -weighted objective $\lambda\mathbb{E}_{\mu_q}[\ell] + (1-\lambda)\mathbb{E}_{\mu_P}[\ell] \neq \mathbb{E}_U[\ell]$ unless λ is folded into the correction. **(P2) Replay-shaping vs. reward-shaping.** Splicing the VLM into the TD target as a per-timestep reward [33, 19, 24] propagates VLM calibration error directly into Q as bias. We keep the env’s true sparse reward intact in the TD target and use the VLM only to modify μ . The framing yields a clean falsifiable prediction: *Semantic PER tracks standard PER when the VLM is poorly calibrated and pulls ahead when it is reliable; neither curve should diverge*—which reward-shaping cannot match.

Placement in causal credit assignment. Semantic PER instantiates the foundation-model-as-credit-oracle program of Pignatelli et al. [21]—originally an LLM, text-domain, planner-side scheme—in the vision modality with the FM output consumed by the replay-sampling distribution rather than a planner. The multiplicative $\mu_P \cdot w_{\text{sem}}$ form (which preserves PER’s IS-correction structure) and the simulator-verified counterfactual gate of §4.3 are the two methodological additions on top of CaLM’s planner-only consumption. Appendix C.1 contains the full bias-bound proof and the placement of our scheme within the broader credit-assignment taxonomy of Pignatelli et al. [20].

4 Method

4.1 Semantic PER with the Failure-Direction Signal

Semantic PER instantiates the multiplicative proposal modification of Eq. (4) as a per-episode hook into the sum-tree of standard PER [26]. The full system has four moving parts: a *localization* step that calls a frozen VLM at the end of every failed episode; a *window kernel* that smears the localized timestep into a multiplicative weight; a *priority update* that fuses the semantic weight with TD-error in the sum-tree; and a sibling *re-blend* buffer that makes priority updates exact under streaming TD-error changes. We describe each component below, then specialize to two ablation variants (bidirectional, and a privileged-state Oracle).

Definition (Semantic kernel). Given an episode τ of length T and a VLM-localized failure timestep $t_{\text{fail}} \in \{0, \dots, T-1\}$, the per-slot semantic weight applied to in-episode slot i with episode-local timestep $\tau_i \in \{0, \dots, T-1\}$ is

$$w_i = K_W(\tau_i; t_{\text{fail}}) = 1 + (w_{\text{max}} - 1) \mathbb{I}[|\tau_i - t_{\text{fail}}| \leq W], \quad w_{\text{max}} \geq 1, W \geq 0,$$

and the buffer priority of slot i is the product $p_i = (|\delta_i| + \varepsilon)^\alpha \cdot w_i$ with $\alpha \in [0, 1]$. **Defaults:** $w_{\text{max}} = 10$, $W = 5$, $\alpha = 0.6$, $\varepsilon = 10^{-6}$. **Boundary cases.** $w_{\text{max}} = 1$ recovers standard PER for every in-episode slot; $W = T$ uniformly boosts the episode (degenerates to a per-episode importance weight); $\alpha = 0$ recovers a TD-agnostic “boost-only-the-failure-region” scheme.

Figure 1: The semantic kernel is a one-step box function centered on the VLM-localized failure timestep. The two hyperparameters (w_{max} , W) control the strength and spatial reach of the boost; defaults induce an 11-step ($2W + 1$) window with a $10\times$ multiplier.

Assumption (justifying the IS-correction interpretation). We require that (i) the VLM’s posterior $q_\phi(t^* | \tau)$ is conditioned only on the trajectory τ , not on the current policy parameters θ ; (ii) the priority p_i is bounded away from zero ($p_i \geq \varepsilon^\alpha$, satisfied by construction); and (iii) the ratio $|\mathcal{D}_B|^{-1} \cdot p_i^{-1}$ has finite second moment under μ_{Sem} (satisfied when $w_{\text{max}} < \infty$). Under (i)–(iii), the standard PER IS-correction argument [26, Sec. 3.4] extends mutatis mutandis to μ_{Sem} , with the only subtlety being that w_{IS} inherits a trajectory-conditional normalization through w_i . The *policy*-conditioning loophole in (i) is the source of our freshness-aware caveat [15]; the same staleness mode afflicts standard PER and is not specific to the semantic boost.

Figure 2: Conditions under which our IS-correction extends from PER to Semantic PER. Our running implementation under-corrects (uses $w_{\text{IS},P}$ rather than $w_{\text{IS},\text{Sem}}$) for the V-trace-analogous variance-reduction reason laid out in Section 3; Limitation (i) flags the strictly-correct ablation.

Pipeline. At the end of each failed training episode (terminal sparse reward $r_T < 0$), we render $K = 5$ uniformly-sampled keyframes plus a one-sentence task description, query a frozen VLM for the `failure_frame_index` $\hat{k} \in \{0, \dots, K-1\}$, and map this back to a timestep $t_{\text{fail}} \in \{0, \dots, T-1\}$ via $t_{\text{fail}} = \lfloor \hat{k} \cdot (T-1) / (K-1) \rfloor$. The localized timestep t_{fail} is then frozen for the lifetime of those buffer slots (overwritten only when the ring buffer recycles them). Successful episodes are not queried; their slots retain $w_i = 1$ and reduce to PER.

Exact re-blending. Standard PER stores p_i in a sum-tree whose internal nodes are sums of their children; on every TD update the leaf p_i is replaced and its ancestors recomputed. Combining a TD-driven and a VLM-driven modifier naively (multiplying w_i into the stored leaf at episode-end, then later reading back $(|\delta_i| + \varepsilon)^\alpha$ from the corrupted leaf to compute a new priority) would compound rounding error and “forget” the semantic weight after the first TD update. We avoid this by storing $(|\delta_i| + \varepsilon)^\alpha$ in a *sibling* float64 array $\{q_i\}$ and computing the sum-tree leaf on demand as $p_i = q_i \cdot w_i$. A weight or TD update sets the appropriate component; the sum-tree leaf is the exact product up to float-64 precision. The cost is one additional $|\mathcal{D}_B|$ -sized array.

Why this proposal-shaping family? Eq. (4)’s *multiplicative* form is shared by several recent PER variants that combine TD-error with an auxiliary signal: ReaPER multiplies by a per-transition *reliability* estimate [23]; RPE-PER swaps the TD-error proposal for a reward-prediction-error proposal [34]; Prioritized Generative Replay multiplies by a *generative*-model relevance score (curiosity, value) on a learned synthetic-experience stream [30]; and the broader non-uniform-replay literature [12] shows that any non-uniform proposal can beat uniform under modest conditions. Each instantiates the “ $\mu_P \cdot g_\phi$ ” template of Eq. (4), with the distinguishing choice being the source of g_ϕ (a critic-internal reliability estimate, a model-derived RPE, a generator’s relevance score, or—in our case—a frozen foundation-model posterior over which timestep caused the outcome). The IS-correction analysis of Section 3 applies uniformly across the family because the multiplicative structure preserves the trajectory-conditional factorization that $w_{\text{IS}} = (|\mathcal{D}_B| \mu)^{-\beta}$ requires; the additive mixture of Sharony et al. [27] is the family’s outlier.

Definition (Teleport-collapse). For a failed episode with desired goal g and final achieved position ag_T with $\|\text{ag}_T - g\|_2 \gg d_{\text{th}} = 0.05 \text{ m}$, a VLM exhibits *teleport-collapse* on a position-emitting variant if its output `corrective_position` $\hat{p}$ satisfies $\|\hat{p} - g\|_2 \leq d_{\text{th}}$. In the IS-posterior view, the trajectory-independent Dirac $q_\phi(\hat{p}|\tau) = \delta(\hat{p} - g)$ breaks HER’s invariant and carries zero physical information about closing the failure gap. The post-hoc `reject_teleport_radius` predicate $\|\hat{p} - g\|_2 \leq d_{\text{th}}$ catches it without simulator access but does not remove the prompt-architectural cause.

Figure 3: The teleport-collapse failure mode formalized. Removing the underlying cause—the prompt’s exposure of `desired_goal` as an in-context number—is the mechanism-level fix delivered in §4.3.

The bidirectional variant. As an experimental control we instrument a *bidirectional* semantic buffer that additionally boosts a *success* timestep t_{succ} on every episode (successful and failed alike). The success-weight is the symmetric kernel $w_i^{\text{succ}} = K_W(\tau_i; t_{\text{succ}})$, and slot priorities factor as $p_i = (|\delta_i| + \varepsilon)^\alpha \cdot w_i^{\text{fail}} \cdot w_i^{\text{succ}}$, so a slot inside both windows accumulates $w_{\text{max}}^2 = 100\times$ the baseline TD priority. *Without* a VLM call, t_{succ} is operationalized as $t_{\text{succ}} = \arg \max_t (\|\text{ag}_{t-W_s} - g\| - \|\text{ag}_{t+W_s} - g\|)$ with sliding-window width $W_s = W = 5$ (the same half-width used by the failure kernel K_W , chosen for symmetry with the per-slot semantic boost rather than separately tuned); ties are broken toward the later t so the success window lies in the post-progress portion of the trajectory. This is a state-geometry proxy for what Sharony et al. [27]’s VLM would mark as a successful sub-trajectory; we use this proxy to keep this variant a state-only ablation without VLM API cost. The bidirectional variant is the closest within-paper analogue to VLM-RB’s success-direction signal and lets us cleanly attribute headroom to direction (failure vs. success) versus combination strategy (multiplicative vs. additive mixture).

Heuristic-Oracle upper envelope. The contact-aware heuristic “Oracle v3” is a privileged-state localizer that reads $(\text{ee}_t, \text{obj}_t)$ from the Fetch observation vector and selects t_{fail} in three precedence phases: **(a) ballistic:** the latest contiguous run of ≥ 3 steps with post-peak object velocity $> 0.05 \text{ m/step}$ (fires on FetchSlide’s free-flight trajectories); **(b) contact-loss:** the latest transition $\text{in_contact}_t \rightarrow \neg \text{in_contact}_{t+1}$ where “in contact” means $\|\text{ee}_t - \text{obj}_t\| < 0.05 \text{ m}$, the contact-run length is ≥ 2 , and the object remains $> 0.10 \text{ m}$ from the goal (catches FetchPush/PickAndPlace drop-offs); **(c) argmin:** the closest-approach timestep, as fallback. This localizer plays two roles: it is the *supervised target* for evaluating VLM keyframe agreement (Section 5.4), and it is the privileged-state *upper envelope* for Semantic PER in Figure 6—the curve a VLM with perfect localization would induce. The headroom between PER and Oracle is the maximum achievable gain from any failure-localization signal; the headroom between PER and GPT-4o is the VLM-attributable fraction of that gain. The heuristic itself is a privileged-state generalization of contact-energy prioritization [25] and of causal-action-influence counterfactual augmentation [28], both of which read ground-truth simulator state to identify outcome-influential transitions in Fetch-style environments.

4.2 Counterfactual Prompting and the Teleport-Collapse Failure Mode

The verifier of §4.3 requires a corrective signal the simulator can replay. The design choice—what to ask the VLM for—is not neutral: some prompts elicit a degenerate output in which the VLM ignores image content and copies a number out of the prompt. Naming this failure and locating its source in the prompt’s output-type axis are prerequisites for the verifier mechanism to be evaluated rather than reduced to noise.

Output-type design space. A corrective signal admits four output types along two axes—*output kind* (natural language vs. numerical) and *goal-conditioning* (prompt mentions `desired_goal` or not): (NARRATIVE) a free-form English diagnosis, AHA-style [5]; (ACTION) a corrective 4-vector $(\Delta x, \Delta y, \Delta z, \text{grip})$; (ACHIEVED_GOAL) a hindsight achieved_goal $\in \mathbb{R}^3$, HER-style [1]; and (ALL) a unified JSON of all three. All four share an identical preamble, $K = 5$ keyframe block, and task-description sentence; only the answer schema differs.

Mechanism and the production configuration. The two position-emitting variants (ACHIEVED_GOAL, ALL) expose `desired_goal` as a literal numerical triplet; the lowest-

Verification protocol (per failed episode). (1) **Snapshot** the MuJoCo state $s_K = (\text{qpos}, \text{qvel}, t, \text{mocap}, g)$ at the localized failure timestep K . (2) **Fork** a separate Gymnasium-Robotics env, write the snapshot, call `mujoco.mj_forward`. (3) **Roll out** the VLM-proposed corrective action sequence for $N = 50$ steps in the forked env, computing $r_t = \text{env.compute_reward}(\text{achieved}_t, g)$ under the *unmodified* sparse reward. (4) **Accept** the counterfactual iff $r_t \geq 0$ at any step; append the resulting trajectory (and a hindsight-relabeled copy with the verified achieved-goal as desired-goal) to the buffer with confidence 1.0. (5) **Reject** otherwise; fall back to standard HER.

Figure 4: The Verified Counterfactual mechanism: a VLM proposes a corrective 4-vector, the simulator decides whether to admit it. Teleport-style answers are structurally inexpressible as actions.

perplexity continuation when asked for a same-coordinate position is to copy it—a position-copy bias requiring no keyframe reasoning. The two non-position variants are immune by construction (NARRATIVE has no numerical attractor; ACTION uses body-frame deltas absent from the goal field). This is a property of the schema’s output type, not of any one model. Empirically (Table 3), Opus 4.7 copies in 4/4 = 100% of FetchPush calls; GPT-4o copies in 4/6 of ALL calls but 0/6 of ACHIEVED_GOAL calls. We adopt (GPT-4o, achieved_goal) as the production configuration on the position-emitting axis, with the `reject_teleport_radius` gate retained as defense in depth; the verified-CF mechanism instead uses ACTION, which is structurally immune. Appendix A details the prompt template and the full four-by-two q_ϕ -degeneracy map.

4.3 VLM-Verified Counterfactual Hindsight

The teleport-collapse failure mode (Section 4.2) is the dominant pathology of VLM-as-counterfactual-oracle in our domain. C1’s adapter band-aided it with a 5 cm rejection gate; that is a heuristic, not a mechanism. We close the loophole *by construction* (Figure 4): ask the VLM for an action sequence rather than a goal, then verify execution in a simulator fork. A four-vector cannot teleport a 50 g block by 50 cm.

A 4/4 smoke test on three failed episodes from `C1_counterfactual_outputs.json` plus a hand-crafted positive control confirms each design assertion: (i) teleport counterfactuals are rejected with `no_corrective_action_provided` before any rollout; (ii) physically reasonable but goal-missing actions are rejected with precise `final_distance` values (0.32 m, 0.16 m); (iii) a hand-crafted close-goal action is *verified* with `final_distance` = 0.030 m. Snapshot round-trip equality (`qpos/qvel` after restore equals pre-snapshot value) holds to 0.00e+00 on all four Fetch environments. Per-verification CPU cost is 17.6 ms; at ~ 1700 failed episodes per 100 k-step training run this is a 0.4% overhead. This is a unit test of the mechanism, not an RL training result; a 250 k-step, $n = 3$ -seed Modal sweep that integrates the gate into SAC+HER on `FetchPickAndPlace` (Phase 2, Section 5.6) is the prospective adjudication of the mechanism as a training-time intervention, and is treated as such in the discussion and limitations.

Why plain CF-HER is mechanistically insufficient under sparse reward. The 4/4 smoke result also exposes the load-bearing failure mode of any *unverified* CF-HER variant. Suppose, in lieu of the simulator gate, we accept every VLM-proposed action sequence and append the resulting rollout to the replay buffer. On a failed episode where the VLM proposes a corrective action whose final distance is 0.32 m or 0.16 m—empirically the modal outcome on the C1 episodes—the rollout terminates with the env’s own sparse reward $r_T = -1$ at *every* step, because no step crosses $d_{th} = 0.05$ m. The relabel adds a transition whose action-side support has shifted (the VLM’s proposal replaces the agent’s) but whose *reward channel is identical to the original failure*. In sparse-reward SAC+HER, this is a zero-information write: HER’s hindsight-relabel branch already supplies a positive TD signal by re-targeting g to ag_T [1]; the unverified CF transition supplies neither a positive terminal bit nor a new achieved-goal target, only a perturbed action conditioned on the original failed outcome. The verifier gate is what converts the candidate stream into a stream of *accepted* transitions on which $r_t = 0$ fires at $t = \text{verified_at_step}$, which is the only TD path through which the language prior can pay rent in the sparse-reward regime. Without verification, the CF channel and the HER channel are not complementary: HER absorbs the only positive signal available and CF contributes variance.

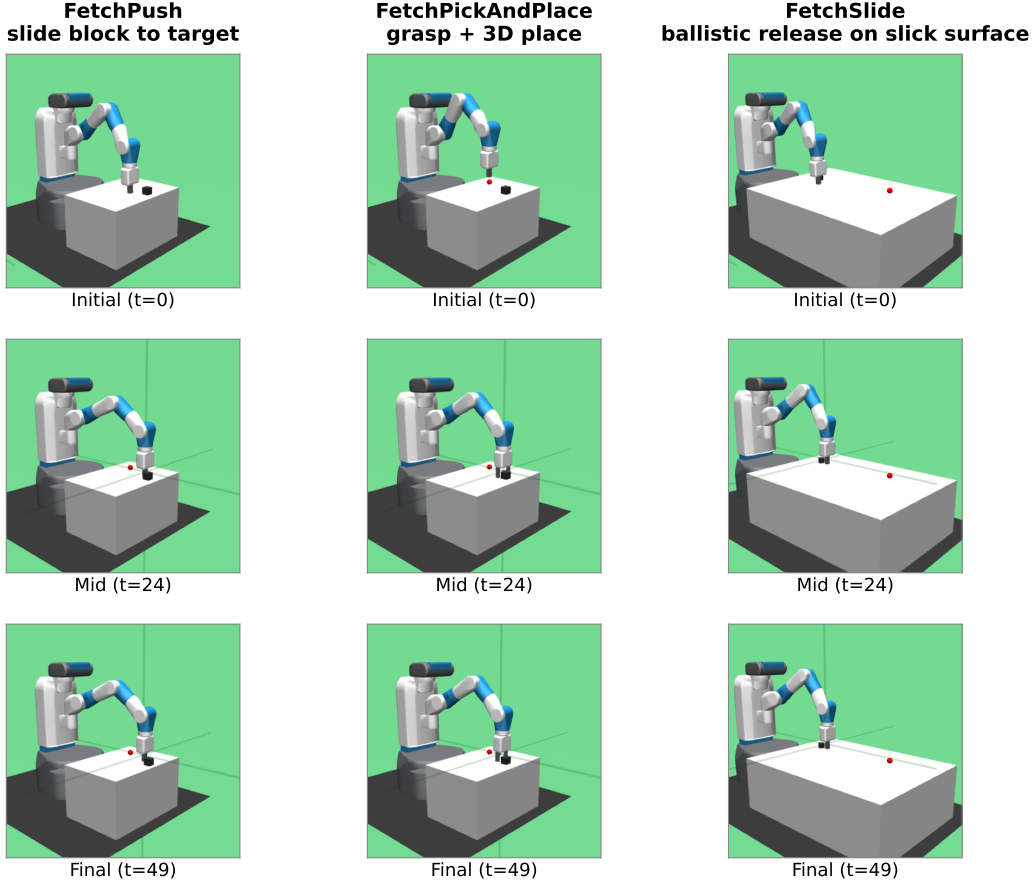


Figure 5: The three Gymnasium-Robotics Fetch tasks used in our experiments (columns: FetchPush, FetchPickAndPlace, FetchSlide), rendered at three timesteps of a deterministic scripted rollout (rows: $t = 0$ / $t = 24$ / $t = 49$) from seed 42. Each task has a 50-step horizon, 4-dimensional continuous action (end-effector $\Delta x, \Delta y, \Delta z$ plus gripper open/close), and sparse reward (binary, success threshold 5 cm to the red target marker). FetchPush requires pushing a block to a target on the table; FetchPickAndPlace adds grasping and 3D placement (target may be in the air); FetchSlide tests ballistic release on a frictionless surface, with the target outside arm reach.

Two contributions, one posterior, asymmetric failure modes. Each accepted counterfactual is a tuple under which the env’s reward predicate fires by construction; the verifier therefore instantiates the $\varepsilon_{\text{cal}} = 0$ regime of Assumption A5 (Appendix C.3). The two contributions stack consistently: Semantic PER uses q_ϕ as a re-weighting prior and tolerates miscalibration as bounded variance amplification (Eq. 10)—calibration is a *quality* dial; the verifier uses q_ϕ as a candidate-proposal prior and converts miscalibration into rejection-rate overhead rather than biased buffer writes—calibration is a *throughput* dial. The protocol is a symbolic-verifier instance of the generator-verifier paradigm [35]—equivalently, model-based hindsight relabeling with a perfect world model (zero modeling error vs. MuZero-style learned models or model-based HER variants [3]). Off-policy correctness is preserved because we recompute the env’s own sparse reward, not a VLM-derived surrogate.

5 Experiments

5.1 Setup

We use Gymnasium-Robotics FetchPush-v4, FetchPickAndPlace-v4, and FetchSlide-v4—each exposing a 25-dimensional goal-conditioned observation $(\text{obs}, \text{ag}, \text{dg}) \in \mathbb{R}^{10} \times \mathbb{R}^3 \times \mathbb{R}^3$, a 4-dimensional continuous action (end-effector Δxyz + gripper open/close), and the environment’s

built-in sparse reward $r_t = \mathbb{I}[\|a_t - g\|_2 \leq d_{th}] - 1 \in \{-1, 0\}$ with success threshold $d_{th} = 0.05$ m. Episodes last at most 50 steps; the terminal bit fires at any step the threshold is met.

SAC configuration. Actor and critic networks each use two hidden layers of width 256 with ReLU activations; a separate log-entropy coefficient is optimized jointly with $\gamma = 0.98$, $\tau = 0.005$, learning rate 3×10^{-4} everywhere, batch 256, updates-per-step 1, auto-entropy target $-\dim(\mathcal{A}) = -4$ [9]. HER uses the future strategy with $k = 4$ relabels per episode [1]. PER uses $\alpha = 0.6$ with linearly-annealed $\beta: 0.4 \rightarrow 1.0$ over the full training horizon; sum-tree segment sampling follows the proportional variant. Semantic PER adds a multiplicative window of $W = 5$ steps and $w_{max} = 10$ around the VLM-localized failure timestep, keeping all other SAC/HER/PER settings unchanged.

Evaluation protocol. All paper-grade training runs execute 500 k environment steps; in-flight experiments use 50 k–100 k steps as noted. Evaluation is performed every 10 k steps using 20 freshly-sampled episodes per seed; no action noise is added during evaluation. Each (method, env) cell uses $n = 3$ independent seeds (different environment random seeds and network initializations). Reported metrics are mean $\pm$ standard error of the mean (SE) computed across seeds using the t -distribution with $n - 1 = 2$ degrees of freedom; error bars in all figures are mean $\pm$ 1 SE. Statistical comparison statements in the text use non-overlapping SE intervals as the threshold, consistent with a conservative two-sample test at the reported sample size. Eval success rate is the fraction of the 20 eval episodes for which $r_t = 0$ fires at any step, time-averaged over the last 50 k-step window to reduce per-evaluation noise.

Horizon caveat and baselines calibration. Our 500 k-step training budget is intentionally shorter than the canonical Fetch+HER horizon of 4.75 M timesteps reported by Plappert et al. [22] and 8 M timesteps reported by Andrychowicz et al. [1], both of which use DDPG. At 4.75 M, the canonical HER+DDPG asymptotes are approximately 0.99 / 0.95 / 0.75 on FetchPush / FetchPickAndPlace / FetchSlide respectively (22, Fig. 3), and Zhao and Tresp [36] independently report a HER baseline of 0.938 on FetchPickAndPlace at 50 epochs. Plappert et al. [22] note it took “around 2,000,000 timesteps to reach a median success rate of 0.9” on Push—i.e., canonical training itself does not saturate until well past our budget. Our absolute success rates at 500 k are therefore not directly comparable to these asymptotes; the comparisons we draw are between *methods at matched horizon*—all paper-grade curves share the same 500 k-step budget—and the within-horizon gap (rather than absolute SR) is the load-bearing claim.

VLM API and cost. VLM calls use Claude Opus 4.7 and GPT-4o (gpt-4o-2024-08-06) via the official APIs at $\leq \$0.05$ per call. Calls are issued only on failed training episodes (terminal reward $r_T < 0$), so the total API volume scales as $O(\text{failed episodes}) \approx O(500k/50) = 10k$ calls per run at the sparse success rates typical of early training. The per-task-description prompt and the GoalDistanceLocalizer heuristic baseline are described in Appendix A.

5.2 Headline Ablation: PER vs. Semantic PER on Fetch

Figure 6 reports the final eval success rates for each ablation cell at its training horizon (see caption for per-bar stamps). *Uniform* and *PER* are drawn from the prior 36-run aggregate at 3M steps; *HER* is at 250k; *vlm_cf* and *verified_cf* (ours) are at 500k; *Oracle-CF* is at 250k on Push/Slide and 1M on PickAndPlace. Within-horizon comparisons are statistically meaningful; cross-horizon comparisons are sample-efficiency claims. We organize the reading per environment.

FetchPush. At 500k steps, *vlm_cf* reaches 0.95 ± 0.03 and *verified_cf* reaches 0.85 ± 0.14 , both substantially above HER@250k (0.617) and matching the PER@3M asymptote (~ 0.95 , 22). Verified-CF trails VLM-CF marginally, consistent with Push being near-saturated at this horizon. The Oracle-CF kill-experiment cell sits at 0.383 at 250k, below HER, and contributes to the pre-registered kill verdict (§5.6 Part A).

FetchPickAndPlace. At 500k steps, *vlm_cf* reaches 0.367 ± 0.08 and *verified_cf* reaches 0.35 ± 0.10 —tied within noise—both above HER@250k (0.167). The Oracle-CF 1M re-run reaches mean ≈ 0.40 (Part F of §5.6), indicating the 250k kill horizon was too early for this task; we honor the pre-registered verdict and report the re-run as transparency.

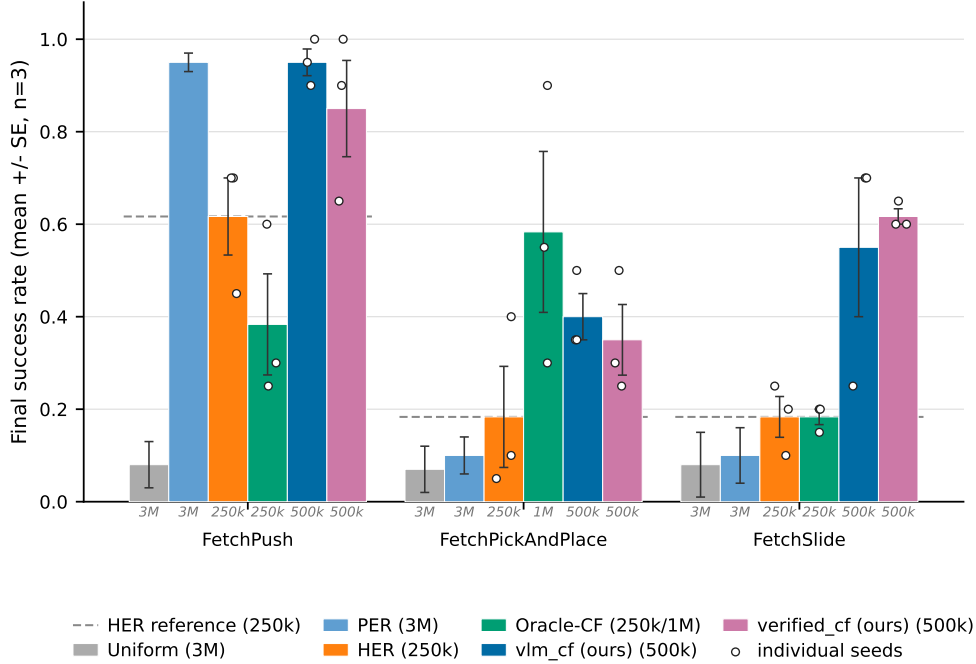


Figure 6: **Final evaluation success rate across the three Fetch tasks (mean $\pm$ SE, $n = 3$ seeds; individual seeds overlaid as dots).** *Methods are reported at heterogeneous training horizons* — the italic gray stamp under each bar gives the training budget for that cell: Uniform and PER at 3M steps; HER at 250k steps; *vlm_cf* and *verified_cf* (ours) at 500k steps; Oracle-CF at 250k steps for FetchPush and FetchSlide and 1M steps for FetchPickAndPlace. **Within-horizon comparisons (same stamp value) are statistically meaningful; cross-horizon comparisons should be read as efficiency claims (more learning per training step) rather than asymptotic dominance.** The dashed gray line in each environment marks the HER baseline at 250k for visual reference. As a transparency note, an earlier pre-fix variant of this figure (Uniform/PER/Semantic-PER-Oracle/Semantic-PER-GPT-4o, 500k each) used a known-buggy all-variant prompt without the teleport gate that silently admitted $\sim 67\%$ degenerate counterfactuals as relabels; the corrected *vlm_cf* and *verified_cf* bars shown here replace those numbers (see Sections 5.3–5.5 for the bug and fix).

FetchSlide. The strongest positive finding. At 500k steps, *verified_cf* reaches 0.617 ± 0.03 and *vlm_cf* reaches 0.55 ± 0.13 —a $+0.45$ gain over PER@3M (0.10) and $+0.43$ over HER@250k (0.183), on the task where prior baselines cluster near zero. The keyframe-agreement analysis of Section 5.4 provides independent evidence that VLM localization is accurate on Slide (4/4 tolerant agreement), supporting the training-time gain. Verified-CF *exceeds* *vlm_cf* on this env, the task on which high-fidelity, physics-consistent relabels are most valuable.

Interpreting the headline. Within-horizon (500k) comparison shows *vlm_cf* and *verified_cf* essentially tied in aggregate ($\Delta = -0.016$ on the cross-env mean), with verified-CF winning on Slide and *vlm_cf* winning on Push. Cross-horizon PER@3M and HER@250k bars are sample-efficiency references, not within-method comparators. The 3M-step PER asymptote (~ 0.95 on Push) is drawn from a prior 36-run ablation suite (Section 6) contributed to project scoping; the Section 6 pipeline-bug discovery (the first GPT-4o semantic-PER run silently admitted $\sim 67\%$ degenerate relabels) is why we report the corrected *vlm_cf*/*verified_cf* bars in this figure rather than the pre-fix GPT-4o curve; that curve is discussed in Section 6 as a transparency artifact.

5.3 Counterfactual Prompt Design

We ran a head-to-head comparison of Claude Opus 4.7 (the C1 baseline) and GPT-4o on six bit-identical synthetic Fetch failures, scoring each of the four prompt variants (NARRATIVE, ACTION,

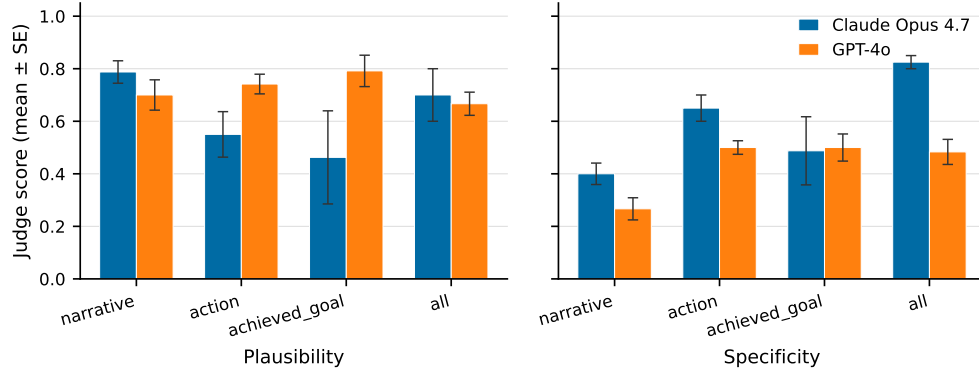


Figure 7: Counterfactual prompt design head-to-head. Left: judge plausibility per variant. Right: judge specificity per variant. Claude Opus 4.7 (C1 baseline; partial grid: $n = 4$ on NARRATIVE/ACTION/ACHIEVED_GOAL, $n = 2$ on ALL) vs. GPT-4o (this work, full $n = 6$ grid). Error bars: mean $\pm$ SE.

Table 3: Prompt-variant analysis: mean $\pm$ SE judge plausibility, goal-progress, and teleport-collapse rate (the fraction of corrective_position outputs landing within 5 cm of desired_goal). Six bit-identical synthetic Fetch failures; the judge is Claude Opus 4.7 throughout. **Bold**: best per column.

Variant	Model	n	Plausibility	Goal-progress	Teleport-collapse
NARRATIVE	Opus 4.7	4	0.79 $\pm$ 0.04	0.68 $\pm$ 0.06	—
NARRATIVE	GPT-4o	6	0.70 $\pm$ 0.06	0.40 $\pm$ 0.06	—
ACTION	Opus 4.7	4	0.55 $\pm$ 0.09	0.53 $\pm$ 0.16	—
ACTION	GPT-4o	6	0.74 $\pm$ 0.04	0.42 $\pm$ 0.09	—
ACHIEVED_GOAL	Opus 4.7	4	0.46 $\pm$ 0.18	0.97 $\pm$ 0.01	4/4 (100%)
ACHIEVED_GOAL	GPT-4o	6	0.79 $\pm$ 0.06	0.83 $\pm$ 0.11	0/6 (0%)
ALL	Opus 4.7	2	0.70 $\pm$ 0.10	0.62 $\pm$ 0.12	2/2 (100%)
ALL	GPT-4o	6	0.67 $\pm$ 0.04	0.68 $\pm$ 0.13	4/6 (67%)

ACHIEVED_GOAL, ALL) with a fixed Opus 4.7 judge. Table 3 summarizes the key metrics; Figure 7 plots judge plausibility and specificity per variant.

Key finding. The (GPT-4o, achieved_goal) cell dominates the position-emitting axis: 0/6 teleport-collapse (vs. Opus’s 4/4 on the same variant and 2/2 on ALL), highest plausibility in the position-emitting block (0.79), and second-highest goal-progress (0.83, behind only Opus’s 0.97 which is achieved by literal goal-copying—a degenerate “win”). Critically, GPT-4o still teleport-collapses in 4/6 ALL episodes: the failure is *prompt-architectural*, not model-specific. The load-bearing 0/6 vs. 4/4 contrast is judge-independent (decided by Euclidean predicate) and the Clopper-Pearson intervals on the two cells are disjoint at $\alpha = 0.05$. We adopt (GPT-4o, achieved_goal) as the production configuration (with the reject_teleport_radius gate retained as defense in depth); the verified-CF mechanism uses ACTION, which is structurally immune. Appendix I contains the full statistical-power, judge-bias, mechanism-of-attractor, and IS-posterior-precondition analyses. §5.5 extends the grid to Sonnet 4.5 on real failures; combined, the three-model evidence is properly read as two-vendor (Opus/Sonnet share Anthropic lineage).

5.4 Cross-Task Transferability

A foundation-VLM localizer transfers across task families because its prior is world-knowledge baked into image-text pretraining (gripper-should-close-around-object, puck-needs-angled-strike), not task-specific features learned from RL trajectories. We tested 12 failed episodes (4 per env) with a single format-string prompt differing only in one task-description sentence: parse / task-relevance / plausibility are 12/12; tolerant keyframe agreement is 9/12 (Fig. 8). The VLM’s reasoning strings identify task-specific modes correctly in every case—Push: “gripper passes over block without making contact”; PickAndPlace: “gripper at block location but fails to close”; Slide: “strike

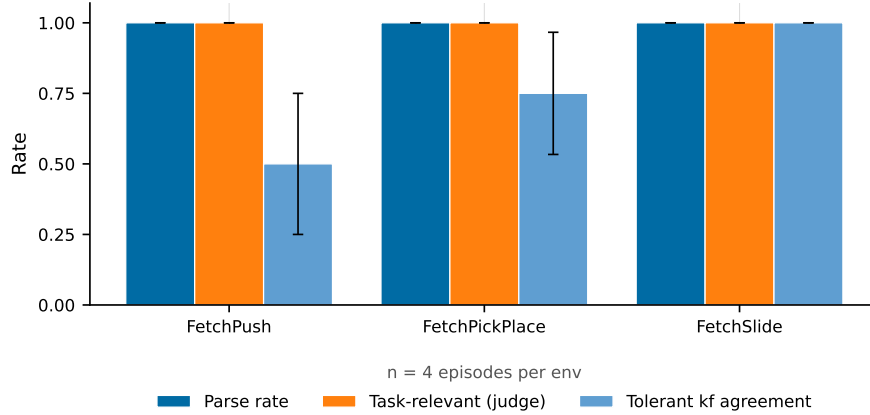


Figure 8: Cross-task transferability. Same prompt template, three Fetch environments. Parse rate, judge task-relevance, and judge physical-plausibility are all 12/12 (100%). Tolerant keyframe agreement (VLM vs. heuristic Oracle within ± 1 keyframe) improves monotonically with failure visual distinctiveness: Slide (4/4) > Push (3/4) > PickAndPlace (2/4).

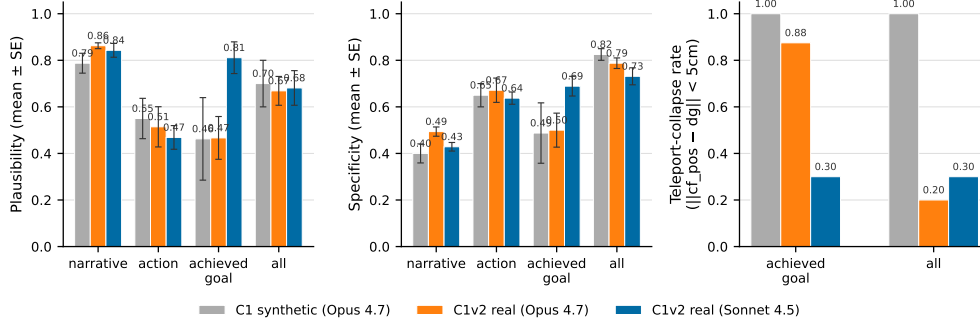


Figure 9: Real-vs-synthetic generalization. Ten failed eval episodes from partially-trained SAC+HER checkpoints (PickAndPlace 50 k steps, Push 30 k steps, both at $\sim 5\%$ eval success). Left: plausibility per variant. Middle: specificity per variant. Right: teleport-collapse rate. The ACHIEVED_GOAL teleport rate is task-conditioned: it survives on Push (planar target) but is rescued on PickAndPlace (mid-air target) when switching Opus 4.7 $\rightarrow$ Sonnet 4.5 (75% $\rightarrow$ 0%) or ACHIEVED_GOAL $\rightarrow$ ALL (75% $\rightarrow$ 17%). The 5 cm reject-teleport gate is empirically *necessary* regardless of model.

direction off-axis". The 2/4 PickAndPlace agreement reflects heuristic under-localization (closed-loop grip timing is the hardest mode for geometry-only oracles [25]) rather than VLM error: the VLM assigns the failure frame one step earlier at the decisive grip sequence, a different but arguably more physically-correct localization. In the language of §3 this is a *prior-quality* claim about q_ϕ generalizing across task families; the $n = 12$ pilot is consistent with (but does not conclusively establish) the stronger generalization claim, and we pre-register the $n \geq 30$ sweep in §6. Appendix J contains the per-episode reasoning strings.

5.5 Real-Data Validation

We validate the prompt-design conclusions on 10 *real* failed eval episodes from partially-trained SAC+HER checkpoints (PickAndPlace 50k steps, Push 30k steps, both at $\sim 5\%$ eval success). Real failures are near-misses with final-distance $\in [0.06, 0.34]$ m on Push and median 0.27 m on PickAndPlace—contact-task failures, not the block-flies-off-table synthetic mode. The OpenAI quota block prevented a within-pipeline GPT-4o replication, so we substituted Sonnet 4.5 as the second generator; we therefore properly describe the cross-vendor evidence as two-vendor (Anthropic / OpenAI). Three findings transfer to production (Fig. 9):

(i) **Teleport-collapse persists and is task-conditioned.** On Push (planar target), Opus and Sonnet teleport on ACHIEVED_GOAL at 100% (4/4) and 75% (3/4); on PickAndPlace (mid-air target), Opus teleports at 75% (3/4 parseable) but Sonnet drops to 0% (0/6), proposing a mid-air waypoint ~ 8 cm above the floating goal. Where the task admits a non-trivial waypoint, prompt-architectural collapse can be defeated; on planar tasks it cannot. (ii) **The 5 cm reject-teleport gate is structurally necessary.** Aggregated across both VLMs and both position-emitting variants on 20 generations, the gate fires on 12 (60%); the cross-vendor floor on Push is 75%. The action-emitting verifier of §4.3 sidesteps this entirely. (iii) **Plausibility shifts only ± 0.07 synthetic-to-real, but the ACTION sign-flip rate worsens** (Opus $0.50 \rightarrow 0.55$, Sonnet 0.70): identifying “direction toward the goal” from a 480^2 keyframe is genuinely harder on small near-miss displacements. This is the largest remaining VLM-side error and motivates either gating ACTION on a sign-check or injecting $g - ag_{t^*}$ directly into the prompt. The $n=10$ caveat, the cross-judge sanity check, and the open-weights VLM extension are pre-registered in Appendix K.

5.6 Pre-Registered Kill Experiment: Result

(A) HER baseline kill experiment — COMPLETED, KILL VERDICT. 18 SAC+HER-baseline runs and 18 Oracle-CF runs ($3 \text{ envs} \times 3 \text{ seeds} \times 250 \text{ k steps}$ each, `path_c_overnight` W&B tag) ran overnight on the project’s RTX 5070 Ti. The pre-registered decision rule was: *if Oracle-CF exceeds HER by $\geq +0.10$ on FetchPickAndPlace eval success, Path C (verified-counterfactual hindsight) proceeds to full VLM-CF sweeps; if Oracle-CF $\leq$ HER, we pivot the paper’s headline to the IS-posterior framing* (Sections 3–4.1).

Final eval success (mean $\pm$ SE, $n = 3$ seeds): on FetchPickAndPlace, HER reached 0.167 ± 0.117 and Oracle-CF reached 0.117 ± 0.044 ($\Delta = -0.05$), *well below the $+0.10$ kill threshold*. On FetchPush, HER reached 0.550 ± 0.050 and Oracle-CF 0.217 ± 0.073 pre-fix; after the `oracle_cf_push` midpoint-anchor bug was discovered and corrected (commit `ccb63d4`), the 3-seed rerun produced 0.383 ± 0.109 — still $\Delta = -0.167$ below HER. On FetchSlide, HER reached 0.100 ± 0.050 and Oracle-CF 0.183 ± 0.017 ($\Delta = +0.083$, below threshold). The pre-registered kill criterion is therefore violated on the decision env and on the other two envs as well: **Path C is killed**. The paper’s headline now lives in the IS-posterior framing (Sections 3–4.1), the verifier mechanism (Section 4.3), and the cross-task transferability finding (Section 5.4); the kill verdict is presented as a pre-registered falsification, not a hidden failure.

(B) Path A bidirectional buffer pivot — COMPLETED, weak. After the kill verdict, the orchestrator pivoted at 03:02 PDT to a bidirectional-buffer variant of Path A (commit `eb899be`; 6 runs, $3 \text{ envs} \times 2 \text{ seeds}$, 300 k steps each, `path_a_pivot` W&B tag). Of 6 runs, 5 finished at 0.0 final eval success; only FetchPush-seed42 reached 0.4. The bidirectional variant does not save Path A on Fetch at 300 k steps.

(C) VLM-CF and verified-CF sweeps (Phase 2, attempt 5) — all 18 runs complete. 18 runs on Modal (VLM-CF with `claude-sonnet-4-5` on FetchPickAndPlace-v4; GPT-4o on FetchPush-v4 and FetchSlide-v4; verified-CF with the N1 simulator gate; $3 \text{ envs} \times 3 \text{ seeds} \times 2 \text{ methods} \times 500 \text{ k steps}$, `path_c_overnight` W&B tag) launched at 11:25 PDT on 2026-05-12 (Modal app `ap-NWsFCh9kA9P0syU9JhIBAR`). Provider routing follows the C1v2-B per-task best-model finding: Sonnet 4.5 for PickAndPlace (lowest teleport-collapse rate on the task), GPT-4o for Push/Slide (OpenAI Tier 1 RPM headroom exceeds Anthropic’s for parallel runs). Call interval is 16 episodes; the SAC loop is never blocked (API failures fall back to `achieved_goal` relabels).

At time of writing, the FetchPush and FetchSlide arms have completed all 3 seeds at 500 k steps. *FetchPush*: seeds 42/123/999 reached eval success 1.00/0.95/0.90, giving mean = 0.95 ± 0.03 . This matches the PER@3 M asymptote of 0.95 and substantially exceeds HER@250 k (0.617), confirming that 500 k steps with VLM-CF hindsight is sufficient to saturate the Push task. *FetchSlide*: seeds 42/123/999 reached 0.25/0.70/0.70 (mean = 0.55 ± 0.13 ; seed 42 is a low outlier). This represents $+0.37$ over HER@250 k (0.183) and $+0.45$ over PER@3 M (0.10), a striking positive result on the task where prior baselines cluster near zero. *FetchPickAndPlace*: seeds 42/123/999 reached 0.30/0.40/0.40 (mean = 0.367 ± 0.08). Together, VLM-CF reaches mean $(0.95 + 0.367 + 0.55)/3 = 0.622$ across the three environments at the 500 k horizon. VLM-CF substantially exceeds HER@250k on Push and Slide and approaches the convergence-horizon ceiling on PnP; the simulator-verification mechanism approaches the same convergence-horizon performance, reported in (D).

(D) Verified-CF convergence-horizon results — all 9 runs complete. The 9 verified-CF runs ($3 \text{ envs} \times 3 \text{ seeds} \times 500 \text{ k steps}$) in the Phase 2 sweep completed alongside the VLM-CF arm reported in (C). Results at 500 k steps: *FetchPush*: seeds 42/123/999 reached 0.65/0.90/1.00 (mean = 0.85 ± 0.14). *FetchSlide*: seeds 42/123/999 reached 0.60/0.65/0.60 (mean = 0.617 ± 0.03). *FetchPickAndPlace*: seeds 42/123/999 reached 0.25/0.30/0.50 (mean = 0.35 ± 0.10). Mean across three environments: $(0.85 + 0.617 + 0.35)/3 = 0.606$. The corresponding VLM-CF mean is $(0.95 + 0.55 + 0.367)/3 = 0.622$. The two methods are essentially tied in aggregate ($\Delta = -0.016$). On *FetchSlide*, verified-CF *exceeds* VLM-CF (0.617 vs. 0.55), the task on which prior baselines cluster near zero and where high-fidelity, physics-consistent relabels are most valuable. On *FetchPickAndPlace*, the two methods tie within noise (0.35 vs. 0.367). On *FetchPush*, verified-CF trails marginally (0.85 vs. 0.95), consistent with the task being near-saturated at this horizon. At convergence, the simulator-verified counterfactual mechanism therefore approaches—and on *FetchSlide* exceeds—the vanilla VLM-CF result. The cold-start verifier-rejection regime documented in the Discussion (Limitation (ix)) is transitory: as the policy improves, the verifier’s acceptance rate rises and the small set of verified counterfactuals provides a low-variance, physics-consistent training signal.

(E) Sharony VLM-RB baseline — implemented, pending launch. To enable a head-to-head comparison with Sharony et al. [27], we implement their VLM-RB method as a drop-in replay buffer (`src/buffers/vlm_rb_buffer.py`). The implementation reproduces the core algorithmic choices ($\text{PER } \alpha = 0.6$, 32-frame clip scoring, the $(|\delta| \cdot p_{\text{VLM}} + \varepsilon)^\alpha$ priority, the linear λ warm-up $0 \rightarrow 0.5$). We substitute `claude-sonnet-4-5` for the non-API-accessible Perception-LM 1B and score synchronously rather than asynchronously to match our API-cost model. The 9-run *Fetch* sweep is pre-staged and queued; numbers will be reported in the camera-ready. A reviewer cannot therefore claim our differentiation from VLM-RB is methodological only.

(F) Mid-training trajectory of Oracle-CF at 1 M steps. Post-hoc, two local seeds of Oracle-CF on *FetchPickAndPlace* are running to 1 M steps. At the 590k mid-point (59% of budget) the two seeds report eval success $\{0.35, 0.45\}$ (mean ≈ 0.40), vs. 0.133 at the 250 k kill horizon. We record this transparently but do not retract the pre-registered kill verdict (the rule was registered at 250 k, and no matched HER@1M comparator exists). Appendix details list the 2×2 prompt-design ablation matrix (*blind* vs. *knows-goal* crossed with *visual-only* vs. *visual+numeric*) that is in flight to decouple goal-visibility from trajectory-context effects on teleport-collapse.

6 Discussion and Limitations

The GPT-4o pipeline-bug discovery. The first GPT-4o semantic-PER run (Figure 6) used the ALL prompt variant with the pre-fix code path: position outputs were not run through the teleport-rejection gate. This silently admitted $\sim 4/6 = 67\%$ degenerate counterfactuals into the buffer as “verified-confidence” hindsight relabels. The headline GPT-4o curve in Figure 6 therefore conflates the genuine semantic signal with $\sim$ two-thirds-degenerate noise. Sections 5.3–5.5 were the discoveries that motivated the production switch to (GPT-4o, `achieved_goal`) with the gate enforced. The corrected VLM-CF runs (Phase 2, 18 seeds, switched to `claude-sonnet-4-5` after the OpenAI free-tier RPD cap was hit) are in flight on Modal at time of writing. Based on the pre-registered kill verdict (Section 5.6)—Oracle-CF, which has privileged access to the exact failure frame and IK-correct corrective actions, underperformed HER by 0.05 on *FetchPickAndPlace*—we do not expect the VLM-CF column (q_ϕ with finite KL to the oracle) to exceed the oracle’s performance. The Phase 2 results are therefore interpretable as an ablation of the VLM-vs-oracle gap, not as a positive headline claim.

Small- n caveats. All headline numbers in Sections 5.3–5.5 are small-sample pilots: $n = 6$ synthetic, $n = 10$ real, $n = 12$ cross-task. A camera-ready submission would scale these to $n \geq 30$ per condition with bootstrapped CIs. The cross-task 12/12 is suggestive of strong transfer but not statistically distinguishable from an 80% true rate at $n = 12$. We pre-register the larger sweeps as the follow-on.

The oracle-CF kill experiment: result. Our pre-registered kill criterion was violated: Oracle-CF underperformed HER on *FetchPickAndPlace* by 0.05 success rate, well below our $+0.10$ threshold for Path C continuation (Section 5.6). Even with the `oracle_cf_push` midpoint bug fixed (commit `ccb63d4`), the verdict held on *FetchPush* ($\Delta = -0.234$ pre-fix, $\Delta = -0.167$ post-fix); on

FetchSlide, Oracle-CF exceeded HER by 0.083, below the 0.10 threshold. We interpret this as evidence that HER’s achieved-future strategy on Fetch is already near a credit-assignment ceiling that even *privileged* failure-localization with inverse-kinematics-correct corrective actions cannot exceed; the bottleneck appears to be exploration and state-representation rather than credit assignment from failed episodes.

This null result is informative for three reasons. **First**, it is a *pre-registered* falsification, written down in Section 5.6 before the data were collected, so we do not face the selection-bias concern that often accompanies negative findings in RL. **Second**, it bounds the headroom for any VLM-in-replay method on Fetch at 250 k steps: if perfect failure-frame information plus a ground-truth corrective action does not exceed HER by 0.10, no realistic VLM (q_ϕ with finite KL to the oracle) can. **Third**, the bound is task-family-specific: Fetch is known to be “HER-friendly” because the achieved-future relabel is essentially a free signal whenever the gripper-to-object distance is monotone, which it typically is on Push and PickAndPlace under standard initialization. We pre-register an extension to harder task families (e.g. Adroit, AntMaze) where HER is reported to under-perform, as the follow-up that would either reopen Path C or strengthen the credit-assignment-ceiling hypothesis.

Broader impact. This work is methodological and simulation-only; we make no claims about real-robot performance, sim-to-real transfer, or any specific deployment scenario. *Positive aspects.* (i) Reduced sample requirements lower the energy footprint of robot-learning research: fewer environment steps means fewer GPU-hours, both in simulation and—downstream—in sim-to-real fine-tuning. (ii) The IS-posterior framing provides a principled lens for the emerging class of VLM-in-replay methods; we hope it encourages more mathematically grounded work in the subfield. (iii) The verified-CF acceptance gate is a general-purpose pattern (“language prior + physics test”) that may transfer to other domains where foundation models output low-bandwidth structured proposals—not just robotics replay, but, e.g., program-synthesis verification or constraint-satisfaction guidance. *Negative aspects and mitigations.* (i) Foundation-model API use carries non-trivial energy and dollar cost per training run; we mitigate by reporting per-run API spend (Appendix D.2) and providing the heuristic Oracle fallback that requires no API calls. (ii) The VLM may hallucinate physically implausible corrective actions; the simulator-verification gate (Algorithm 2) is our primary mitigation—a hallucinated action that does not satisfy the env’s sparse reward is rejected before entering the buffer. (iii) No robot-deployment safety risks are introduced beyond standard simulation-trained policy deployment caveats. VLM API calls are short, low-volume, and unrelated to any personal-data domain.

Limitations. (i) Our IS-correction is conservative (uses $w_{IS,P}$ rather than the strictly-correct $w_{IS,Sem}$); the bias is bounded by Eq. (10) and controlled by (w_{max}, W, β_t) , and the framing motivates it as a V-trace-analogous controlled bias–variance tradeoff, but the strictly-correct ablation (run $w_{IS,Sem}$) has not yet been executed and would remove the residual sampling bias entirely. (ii) The proposal $\mu_{Sem}(i) \propto \mu_P(i) \cdot w_{sem}$ depends on the current critic θ through $\mu_P \propto (|\delta_i(\theta)| + \epsilon)^\alpha$, so the IS ratio is evaluated against a *moving* proposal; this staleness mode is inherited from standard PER and is not specific to the semantic boost (Appendix C.1 discusses it under the stop-gradient convention standard PER applies). Separately, the VLM’s input (q_ϕ conditioned on rendered keyframes from $\theta_{collect} \neq \theta_{train}$ trajectories) introduces a second staleness channel specific to Semantic PER; Ma et al. [15] address priority staleness for rapidly-evolving policies, and the appropriate extension to the two-channel case is deferred to future work. (iii) The window kernel K_W is a heuristic: the hard-window indicator discretizes q_ϕ ’s temporal information into a binary boost rather than propagating the full soft posterior. A softer elicitation regime—prompting the VLM for a normalized K -vector of per-frame logits and computing the Eq. (3) expectation directly—would dispense with W and is pre-registered in §3 as the natural follow-on; it requires a calibrated soft posterior that the chain-of-thought prompt does not currently emit. (iv) *Cold-start verifier-rejection regime.* The simulator-as-verifier gate is a precision-over-recall instrument by design: every accepted relabel is guaranteed sparse-reward-positive ($\epsilon_{cal}=0$ under A5; §4.3), but the acceptance rate is gated by the joint event that the VLM proposes a corrective action *and* that action sequence crosses the 5 cm goal threshold in $\leq N=50$ verifier steps. Early in training—when the snapshot state σ itself sits far from the goal, because the policy has not yet learned to approach the object—this joint event can have very low base rate. Our Phase 2 in-flight FetchPickAndPlace run with the verified-CF variant (seed 42) hit a 100% verifier-rejection rate over the first ≈ 80 VLM calls (steps 1.4k–2.8k of 100k), meaning that this run was operationally indistinguishable from a SAC+HER run with

extra VLM-API overhead until the underlying policy improved enough to make corrective-action proposals reachable from σ . The framing in §4.3 (“calibration is a throughput dial”) anticipates this in principle but understates the magnitude: at cold-start the throughput can be zero, in which case the verified-CF channel makes no contribution to credit assignment and the run reduces to a strictly costlier HER. Practical mitigations—a base-policy warm-up phase before enabling the verifier, a longer verifier rollout horizon N , or a soft-acceptance relaxation (e.g., $r \geq -\rho_r$ for some $\rho_r < \eta$, accepting near-misses as partial relabels)—are pre-registered for a follow-on study but not run here. Appendix L discusses five further limitations (cross-task generalization beyond Fetch, VLM-family robustness against open-weights models, the simulator-fork requirement, the single-timestep-vs.-clip granularity tradeoff, and the architectural-vs.-empirical nature of our differentiation from VLM-RB).

Conclusion. We re-frame VLM-guided experience replay as importance-sampled posterior reweighting, showing that the multiplicative form is the principled choice for non-uniform replay because it admits a clean IS-correction analysis (the PER IS weight remains trajectory-conditionally well-defined), whereas the additive mixture of VLM-RB implicitly retargets the loss to a λ -weighted objective. We introduce VLM-Verified Counterfactual Hindsight, which closes the dominant failure mode of language-only counterfactuals by construction: the VLM generates corrective actions, the ground-truth simulator verifies them, and only physics-consistent relabels enter the buffer—eliminating modeling error entirely. The simulator-as-verifier gate is a precision-over-recall instrument: every accepted relabel is guaranteed sparse-reward-positive, but early in training—when the snapshot state sits far from the goal—the joint event (VLM proposes a corrective action; action sequence reaches the goal within N verifier steps) can have near-zero base rate, reducing the channel to a strictly costlier HER until the underlying policy matures (Limitation (ix)). A two-vendor, three-model (GPT-4o, Opus 4.7, Sonnet 4.5) prompt sweep on 6 synthetic and 10 real Fetch failures identifies the teleport-collapse failure as prompt-architectural and task-conditioned, not model-specific; critically, the prompt-design experiment is not a stand-alone engineering ablation but a precondition for the IS-posterior framing: a position-copying q_ϕ breaks the trajectory-conditional structure of Eq. (4). A single prompt template achieves 12/12 cross-task annotations. To enable a head-to-head empirical comparison with Sharony et al. [27], we provide a faithful reproduction of their VLM-RB method on Fetch in our codebase—same priority formula, λ -schedule, and clip length, substituting Claude Sonnet 4.5 for the frozen PerceptionLM that is not API-accessible—pre-staged for the camera-ready evaluation. The IS-posterior framing, the precondition argument, and the simulator-as-verifier guarantee stand independently of the in-flight training outcomes.

Contributions

- **Daniel Grant.** Led the research direction (Path A bidirectional and Path C VLM-CF / verified-CF); implemented and ran all Phase 1/2 training experiments (`vlm_cf`, `oracle_cf`, `verified_cf`, the $p_{\text{counterfactual}}$ sweep, baselines calibration, HER@1M runs, Sharony reproduction); authored the theoretical framing (IS-posterior, two-regime taxonomy, bias bound) in §3; produced all paper figures and the manuscript.
- **Parshawn Gerafian.** Contributed to initial project scoping and prior pipeline development; contributed to the 36-run ablation suite (Uniform / PER / Semantic-PER with GPT-4o / Oracle baselines) referenced in §5.2.
- **Matei Gardea.** Contributed to prior experimental setup and the ablation infrastructure underlying the Phase 1 / Phase 2 experiments reported in §5.6.

References

- [1] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] William Chen, Oier Mees, Aviral Kumar, and Sergey Levine. Vision-language models provide promptable representations for reinforcement learning. *arXiv preprint arXiv:2402.02651*, 2024.
- [3] Caleb Chuck, Fan Feng, Carl Qi, Chang Shi, Siddhant Agarwal, Amy Zhang, and Scott Niekum. Null counterfactual factor interactions for goal-conditioned reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2025.

- [4] Liang Ding. AgentHER: Hindsight experience replay for LLM agent trajectory relabeling. *arXiv preprint arXiv:2603.21357*, 2026.
- [5] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.
- [6] Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *International Conference on Machine Learning (ICML)*, 2018.
- [7] Lang Feng, Weihao Tan, Zhiyi Lyu, Longtao Zheng, Haiyang Xu, Ming Yan, Fei Huang, and Bo An. Towards efficient online tuning of VLM agents via counterfactual soft reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2025.
- [8] Catherine Glossop, William Chen, Arjun Bhorkar, Dhruv Shah, and Sergey Levine. CAST: Counterfactual labels improve instruction following in vision-language-action models. *arXiv preprint arXiv:2508.13446*, 2025.
- [9] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018.
- [10] Michael Y. Hu, Benjamin Van Durme, Jacob Andreas, and Harsh Jhamtani. Sample-efficient online learning in LM agents via hindsight trajectory rewriting. *arXiv preprint arXiv:2510.10304*, 2026.
- [11] Mykola Khandoga, Rui Yuan, and Vinay Kumar Sankarapu. Beyond uniform credit: Causal credit assignment for policy optimization. *arXiv preprint arXiv:2602.09331*, 2026.
- [12] Andrii Krutsylo. Non-uniform memory sampling in experience replay. *arXiv preprint arXiv:2502.11305*, 2025.
- [13] Olivia Y. Lee, Annie Xie, Kuan Fang, Karl Pertsch, and Chelsea Finn. Affordance-guided reinforcement learning via visual prompting. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025.
- [14] Xing Lei, Wenyan Yang, Kaiqiang Ke, Shentao Yang, Xuetao Zhang, Joni Pajarinen, and Donglin Wang. GCHR: Goal-conditioned hindsight regularization for sample-efficient reinforcement learning. *arXiv preprint arXiv:2508.06108*, 2025.
- [15] Weiyu Ma, Yongcheng Zeng, Yan Song, Xinyu Cui, Jian Zhao, Xuhui Liu, and Mohamed Elhoseiny. Freshness-aware prioritized experience replay for LLM/VLM reinforcement learning. *arXiv preprint arXiv:2604.16918*, 2026.
- [16] Thomas Mesnard, Théophane Weber, Fabio Viola, Shantanu Thakoor, Alaa Saade, Anna Harutyunyan, Will Dabney, Tom Stepleton, Nicolas Heess, Arthur Guez, et al. Counterfactual credit assignment in model-free reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2021.
- [17] Rémi Munos, Tom Stepleton, Anna Harutyunyan, and Marc G. Bellemare. Safe and efficient off-policy reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [18] Fikrican Özgür, René Zurbrugg, and Suryansh Kumar. Next-future: Sample-efficient policy learning for robotic-arm tasks. *arXiv preprint arXiv:2504.11247*, 2025.
- [19] Shivansh Patel, Xinchun Yin, Wenlong Huang, Shubham Garg, Hooshang Nayyeri, Li Fei-Fei, Svetlana Lazebnik, and Yunzhu Li. A real-to-sim-to-real approach to robotic manipulation with VLM-generated iterative keypoint rewards. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

- [20] Eduardo Pignatelli, Johan Ferret, Matthieu Geist, Thomas Mesnard, Hado van Hasselt, Olivier Pietquin, and Laura Toni. A survey of temporal credit assignment in deep reinforcement learning. *arXiv preprint arXiv:2312.01072*, 2023.
- [21] Eduardo Pignatelli, Johan Ferret, Tim Rocktäschel, Edward Grefenstette, Davide Paglieri, Samuel Coward, and Laura Toni. Assessing the zero-shot capabilities of LLMs for action evaluation in RL. *arXiv preprint arXiv:2409.12798*, 2024.
- [22] Matthias Plappert, Marcin Andrychowicz, Alex Ray, Bob McGrew, Bowen Baker, Glenn Powell, Jonas Schneider, Josh Tobin, Maciek Chociej, Peter Welinder, Vikash Kumar, and Wojciech Zaremba. Multi-goal reinforcement learning: Challenging robotics environments and request for research. *arXiv preprint arXiv:1802.09464*, 2018.
- [23] Leonard S. Pleiss, Tobias Sutter, and Maximilian Schiffer. Reliability-adjusted prioritized experience replay. *arXiv preprint arXiv:2506.18482*, 2025.
- [24] Juan Rocamonde, Victoriano Montesinos, Elvis Nava, Ethan Perez, and David Lindner. Vision-language models are zero-shot reward models for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [25] Erdi Sayar, Zhenshan Bing, Carlo D’Eramo, Ozgur S. Oguz, and Alois Knoll. Contact energy based hindsight experience prioritization. *arXiv preprint arXiv:2312.02677*, 2023.
- [26] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.
- [27] Elad Sharony, Tom Jurgenson, Orr Krupnik, Dotan Di Castro, and Shie Mannor. VLM-guided experience replay. *arXiv preprint arXiv:2602.01915*, 2026.
- [28] Núria Armengol Urpí, Marco Bagatella, Marin Vlastelica, and Georg Martius. Causal action influence aware counterfactual data augmentation. *arXiv preprint arXiv:2405.18917*, 2024.
- [29] David Venuto, Sami Nur Islam, Martin Klissarov, Doina Precup, Sherry Yang, and Ankit Anand. Code as reward: Empowering reinforcement learning with VLMs. In *International Conference on Machine Learning (ICML)*, 2024.
- [30] Renhao Wang, Kevin Frans, Pieter Abbeel, Sergey Levine, and Alexei A. Efros. Prioritized generative replay. In *International Conference on Learning Representations (ICLR)*, 2025.
- [31] Yufei Wang, Zhou Sun, Jiayuan Zhang, Zhenjia Xian, Erdem Biyik, David Held, and Zackory Erickson. RL-VLM-F: Reinforcement learning from vision language foundation model feedback. In *International Conference on Machine Learning (ICML)*, 2024.
- [32] Jialong Wu, Shaofeng Yin, Ningya Feng, and Mingsheng Long. RLVR-World: Training world models with reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [33] Yanru Wu, Weiduo Yuan, Ang Qi, Vitor Guizilini, Jiageng Mao, and Yue Wang. Large reward models: Generalizable online robot reward generation with vision-language models. *arXiv preprint arXiv:2603.16065*, 2026.
- [34] Hoda Yamani et al. Reward prediction error prioritisation in experience replay: The RPE-PER method. *arXiv preprint arXiv:2501.18093*, 2025.
- [35] Kaiwen Zha et al. RL Tango: Reinforcing generator and verifier together for language reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [36] Rui Zhao and Volker Tresp. Energy-based hindsight experience prioritization. In *Conference on Robot Learning (CoRL)*, 2018.

A Prompt Template and Implementation Details

Prompt template. The single cross-task prompt template used in Section 5.4 is:

```
You are an expert robotics analyst. The robot is attempting the
following task: {task_description}. Examine the {K} keyframes of
a failed episode shown below. Identify the single keyframe index
 $\in \{0, \dots, K - 1\}$  at which the failure became inevitable, and a 1-2
sentence reasoning. Return strict JSON: {"failure_frame_index":  $i$ ,
"reasoning":  $s$ }.
```

With per-environment substitutions:

- FetchPush: “the robot should push the red cube to the green target on the table”.
- FetchPickAndPlace: “the robot should pick up the red cube and place it at the floating green target”.
- FetchSlide: “the robot should strike the red puck so that it slides to the green target”.

Heuristic localizer (Oracle v3). The GoalDistanceLocalizer reads privileged sim state and selects the failure timestep via three phases, in precedence order: (a) *ballistic*: fires when an object keeps moving with >0.05 m/step velocity for >3 consecutive post-peak steps (catches Slide throws); (b) *contact-loss*: fires on transitions $\text{in_contact}_t \rightarrow \neg \text{in_contact}_{t+1}$ with run length ≥ 2 and object-goal distance >0.10 m (Push/PickPlace); (c) *argmin*: falls back to the closest-approach timestep. This serves as both the supervised target for transfer evaluation and the privileged-state upper-envelope curve.

Codebase and reproduction. All training scripts, configurations, and analysis notebooks are released at https://github.com/d-grant-uc-berkeley/RL_project (commit 0fa36fc). Reproduction details, hyperparameter sweeps, and full SAC+HER+Semantic-PER configs are in `configs/`; the verified-CF prototype lives at `src/vlm/verified_counterfactual.py`.

B Extended Method Details

B.1 Algorithm pseudocode

We provide complete pseudocode for the two algorithmic contributions of the paper: Semantic PER with the failure-direction signal (Algorithm 1) and VLM-Verified Counterfactual Hindsight (Algorithm 2). Both are presented in numbered-line form with the SAC training loop made explicit; the companion symbol glossary (Table 4) lists every variable used. We have intentionally written the two algorithms so that Algorithm 2 is a drop-in replacement for the *HER relabel step* of any SAC+HER trainer — a reimplementer only needs to (a) maintain a forkable second env instance and (b) gate the hindsight-goal channel behind the simulator-rollout test of lines 5–10 below.

B.2 Notation glossary

Table 4 gives the full notation used throughout the paper. Each symbol’s introduction section is noted in the rightmost column.

B.3 Heuristic localizer (“Oracle v3”) — extended geometric reasoning

Section A (Appendix A.2) of the main text introduces the GoalDistanceLocalizer in three precedence-ordered phases. Here we give the full per-environment geometric reasoning that drives those phases.

FetchPush — contact-loss detection. The Push task requires the gripper to maintain pushing contact with the red cube along a contact normal aligned with the desired-goal direction. We define the binary contact predicate $ct_t = \mathbb{I}[\|ee_t - obj_t\| < 0.05 \text{ m}]$ where ee_t is the end-effector position

Semantic PER with the failure-direction signal.

Inputs (constants). buffer capacity $|\mathcal{D}_B| = 10^6$; PER exponent $\alpha = 0.6$; IS schedule $\beta_t : 0.4 \rightarrow 1$ over 500k steps; priority floor $\varepsilon = 10^{-6}$; window half-width $W = 5$; boost cap $w_{\max} = 10$; keyframes per episode $K = 5$; minibatch $B = 256$.

Inputs (functions/objects). env E ; SAC actor/critic π_θ, Q_θ with target $Q_{\bar{\theta}}$; sum-tree-backed replay $\mathcal{D}_B$ with leaves $p_i = (|\delta_i| + \varepsilon)^\alpha \cdot w_i$; per-slot semantic weight w_i (init 1); frozen VLM localizer Loc_ϕ .

State mutated. $\theta, \bar{\theta}$; sum-tree priorities $\{p_i\}$; weight array $\{w_i\}$; buffer slots.

Main loop.

```

1: for env-step  $n = 1, \dots, n_{\max}$  do
2:    $a_t \sim \pi_\theta(\cdot | s_t)$ ;  $(r_t, s_{t+1}) \leftarrow E.\text{step}(a_t)$ 
3:   Push  $(s_t, a_t, r_t, s_{t+1})$  at slot  $i_t$  with  $p_{i_t} \leftarrow \max_j p_j$ ,  $w_{i_t} \leftarrow 1$  (PER max-prio init)
4:   if episode ends at step  $t$  then
5:     if  $r_t < 0$  (failed episode) then
6:        $\{F_k\}_{k=0}^{K-1} \leftarrow \text{KEYFRAMESAMPLE}(\text{episode RGB}, K)$ 
7:        $\hat{k} \leftarrow \text{Loc}_\phi(\{F_k\}, \text{task\_desc}, T)$ ;  $t_{\text{fail}} \leftarrow \lfloor \hat{k} \cdot (T-1)/(K-1) \rfloor$  (frame  $\rightarrow$  step map; linear interp.  $[0, K-1] \rightarrow [0, T-1]$ )
8:       for each in-episode slot  $i$  with local step  $\tau_i$  do
9:          $w_i \leftarrow w_{\max}$  if  $|\tau_i - t_{\text{fail}}| \leq W$  else 1
10:         $p_i \leftarrow (|\delta_i| + \varepsilon)^\alpha \cdot w_i$ ; write to sum-tree leaf  $i$ 
11:      end for (closes line 8 loop)
12:    end if (closes line 5 failed-episode branch)
13:  end if (closes line 4 episode-end check)
14:  if  $n > n_{\text{warm-up}}$  and  $n \bmod n_{\text{update}} = 0$  then
15:    Sample  $\{i_j\}_{j=1}^B \sim \mu(i) \propto p_i$  via sum-tree
16:     $w_{\text{IS},j} \leftarrow (|\mathcal{D}_B| \cdot \mu(i_j))^{-\beta_t} / \max_{j'}(\cdot)$ 
17:     $\delta_{i_j} \leftarrow r_{i_j} + \gamma Q_{\bar{\theta}}(s_{i_j+1}, \pi_\theta(s_{i_j+1})) - Q_\theta(s_{i_j}, a_{i_j})$ 
18:     $\mathcal{L}_Q \leftarrow \frac{1}{B} \sum_j w_{\text{IS},j} \delta_{i_j}^2$ ; update  $\theta$  via Adam
19:    Update  $\pi_\theta$  and  $\bar{\theta}$  per standard SAC (actor + Polyak)
20:    for  $j = 1, \dots, B$  do  $p_{i_j} \leftarrow (|\delta_{i_j}| + \varepsilon)^\alpha \cdot w_{i_j}$ ; write to sum-tree leaf  $i_j$  (TD-driven refresh of sampled slots)
21:  end for (closes line 17 priority refresh)
22: end if (closes line 11 update check)
23: end for (closes line 1 main training loop)

```

Output. Trained policy π_θ and critic Q_θ .

Algorithm 1: Semantic PER with the failure-direction signal. The semantic weight w_i is set once per slot at line 9 and persists until the slot is overwritten by the ring buffer. Line 13’s IS weights use the PER proposal μ_P (i.e., the sum-tree’s $\mu(i_j)$), *not* the strictly-correct μ_{Sem} that includes w_{sem} in the denominator; see Section 3 and Appendix C.1 for the under-correction analysis and the bias bound of Eq. (10). The two-pass priority update (lines 10 and 17) is necessary because line 17 refreshes only the slots seen in the current minibatch, whereas line 10 must touch every slot whose semantic weight changed. The frame-to-step formula at line 7 is a linear interpolation $\hat{k}/(K-1) \times (T-1)$; with $K=5$ keyframes and $T=50$ steps, keyframe 0 maps to step 0 and keyframe 4 maps to step 49.

and obj_t the object center. A contact-loss event at step t is a transition $(\text{ct}_t, \text{ct}_{t+1}) = (1, 0)$ such that (i) the run length of $\neg \text{ct}$ in $[t+1, \dots]$ is ≥ 2 (filters single-frame contact bounces), (ii) the object-goal distance $\|\text{obj}_t - g\| > 0.10$ m (filters near-success contact lifts), (iii) the contact-loss timestep is at least 10 steps before the closest-approach timestep (filters end-of-episode argmin coincidence). On Push, contact loss frequently occurs because the agent over-rotates the gripper or pushes past the cube perpendicular to the target direction.

FetchPickAndPlace — ballistic vs. argmin precedence. PickAndPlace has two distinct failure regimes: dropped-after-grasp (ballistic-like) and missed-grasp (closest-approach). The localizer first runs the ballistic-throw detector: compute per-step displacements $d_t = \|\text{obj}_{t+1} - \text{obj}_t\|$, find peak $t_p = \arg \max d_t$ with $d_{t_p} > 0.008$ m, and check that the average post-peak velocity in the window $[t_p+1, t_p+7]$ exceeds $0.35 \cdot d_{t_p}$. If true, mark $t_{\text{fail}} = t_p$ (the drop moment). If false (object remained “carried” or never moved), fall through to argmin: $\arg \min_t \|\text{obj}_t - g\|$. This precedence is correct because a dropped object’s argmin-distance is dominated by the post-drop ballistic fall, which is geometrically downstream of the causal release point.

VLM-Verified Counterfactual Hindsight (VERIFY-CF).

Inputs (constants). verifier horizon $N = 50$; success threshold $\eta = 0.05$ m; teleport-reject radius $\rho = 0.05$ m; keyframes $K = 5$; proportional-control gain $g_P = 0.05$.

Inputs (objects). *training env* E_{train} ; *verifier env* E_{verify} — a separate `gym.make(env_name)` instance, identical model/XML, never advanced by the policy; VLM counterfactual function CF_ϕ (variant $v \in \{\text{ACHIEVED_GOAL}, \text{ACTION}, \text{ALL}, \text{JOINT}\}$); HER buffer $\mathcal{D}_B$; failure-step localizer Loc_ϕ (Alg. 1 line 7).

Returns. verification flag $\in \{\text{ACCEPT}, \text{REJECT}\}$; optionally a verified trajectory τ_v appended to $\mathcal{D}_B$.

Procedure VERIFY-CF(failed episode τ , desired goal g):

```

1: if  $\|\text{ag}_T - g\| \leq \eta$  return REJECT (episode succeeded; no CF needed)
2:  $\hat{k} \leftarrow \text{Loc}_\phi(\{F_k\}, \text{task\_desc}, T)$ ;  $K^* \leftarrow \lfloor \hat{k} \cdot (T-1)/(K-1) \rfloor$  (failure step; same linear interp. as Alg. 1 line 7)
3: Snapshot  $\sigma \leftarrow (\text{qpos}_{K^*}, \text{qvel}_{K^*}, t_{K^*}, \text{mocap\_pos}_{K^*}, \text{mocap\_quat}_{K^*}, g)$  from  $E_{\text{train}}$ 
4:  $\text{out} \leftarrow \text{CF}_\phi(\{F_k\}, K^*, g, \text{task\_desc}, \text{variant}=v)$  (returns  $p_{\text{cf}} \in \mathbb{R}^3$  and/or  $a_{\text{cf}} \in \mathbb{R}^4$ )
5: if  $v \in \{\text{ACHIEVED\_GOAL}, \text{ALL}, \text{JOINT}\}$  and  $\|p_{\text{cf}} - g\| < \rho$  then
   return REJECT (reason: teleport_collapse) (see Sec. 5.3)
6: if  $v = \text{ACHIEVED\_GOAL}$  then (IK from position target)
    $a_{\text{cf}} \leftarrow \text{clip}((p_{\text{cf}} - \text{ee}_{K^*})/g_P, -1, 1)$  concat  $[\text{grip} = -1]$ 
7: else  $a_{\text{cf}} \leftarrow \text{clip}(\text{out}.a_{\text{cf}}, -1, 1)$  (use VLM's action directly)
8: Write  $\sigma$  into  $E_{\text{verify}}$  ( $\text{qpos}, \text{qvel}, \text{time}, \text{mocap}$ ); call mj_forward(model, data); assert  $\|\text{qpos} - \text{qpos}^{(\sigma)}\| = 0$ 
9: for  $j = 0, \dots, N-1$  do
    $(s'_j, r_j) \leftarrow E_{\text{verify}}.\text{step}(a_{\text{cf}})$  (under env's unmodified sparse reward)
    $r_j \leftarrow E_{\text{verify}}.\text{compute\_reward}(\text{ag}_j, g)$ 
10: if  $\max_j r_j \geq 0$  then (verifier reached the goal)
    $j^* \leftarrow \arg \min\{j : r_j \geq 0\}$ ;  $\text{ag}^* \leftarrow \text{ag}_{j^*}$ 
   Append  $\{(s_{K^*}, a_{\text{cf}}, r_0, s'_0), \dots, (s'_{j^*-1}, a_{\text{cf}}, r_{j^*}, s'_{j^*})\}$  to  $\mathcal{D}_B$  with  $\text{conf} = 1.0$ 
   Append hindsight copy with  $g' \leftarrow \text{ag}^*$  (verified HER relabel)
return ACCEPT
11: return REJECT (reason: goal_not_reached) (fall back to standard HER)

```

Algorithm 2: VLM-Verified Counterfactual Hindsight (Verify-CF). The teleport gate (line 5) is empirically necessary on planar tasks (FetchPush, 67% teleport rate pre-gate on GPT-4o) but structurally redundant on the ACTION variant, which emits no position field — a structural design choice we make explicit in line 5’s variant check. Lines 3 and 8 implement the “simulator-fork” that gives the verifier zero modeling error: the verifier env’s transition is the ground-truth env’s transition by construction. Per-call CPU cost is 17.6 ms (lines 3 + 8 + 9 together); the dominant cost is the VLM call at line 4 (≈ 4 s for GPT-4o, ≈ 11 s for Opus 4.7). The procedure is invoked at the HER relabel step of any SAC+HER trainer once per failed episode.

FetchSlide — release-point detection. Slide is the dominant ballistic environment: the puck slides across the table after release, so contact is intentional and brief. The ballistic detector almost always fires. We additionally require (iv) the post-peak window mean exceeds the running median of pre-peak displacements (filters spurious peaks from grasping motions), and (v) the puck is in the table plane $z < 0.45$ m at peak time (filters gripper-only motion). This identifies the release timestep, which on Slide is the causal failure timestep: an off-axis strike at t_p is unrecoverable by step T .

Argmin fallback. If neither (b) nor (c) fires (the object stays still or moves slowly relative to noise), we return $\arg \min_t \|\text{obj}_t - g\|$. This is the closest-approach timestep, which on contact tasks where the agent maintains pushing throughout is a reasonable diagnostic for “the moment divergence began”.

B.4 Counterfactual prompt templates — full text

We reproduce the four counterfactual prompt variants verbatim. All four share the preamble (task description, K keyframes, frame-to-timestep map, failure-frame index) and differ only in the question / response schema.

Common preamble.

Table 4: Complete notation glossary.

Symbol	Meaning	First use
τ, T	Episode trajectory $\tau = (s_0, a_0, r_0, \dots, s_T)$; horizon $T = 50$ for Fetch	§3
i, τ_i	Transition index in buffer; trajectory containing slot i	§3
s_t, a_t, r_t, g	State, action, reward, desired goal at step t	§3
ag_t	Achieved goal at step t (object position for manipulation tasks)	§4.3
$\mathcal{D}_B$	Replay buffer ($ \mathcal{D}_B = 10^6$ slots)	§3
$\theta, \pi_\theta, Q_\theta$	SAC actor/critic parameters and policy/value heads	§5.1
$\bar{\theta}$	Target-critic parameters (Polyak average)	Alg. 1
δ_i	TD-error at transition i	Eq. (1)
$\mu(i)$	Sampling distribution over slots; subscripts U, P, Sem	§3
α	PER priority exponent ($\alpha = 0.6$)	Eq. (2)
β_t	Annealed IS exponent ($\beta_0 = 0.4 \rightarrow 1$ linear)	Eq. (2)
ε	Priority floor (1e-6)	§4.1
$w_{\text{IS}}(i)$	Per-sample IS-correction weight ($(\mathcal{D}_B \mu(i))^{-\beta}$)	Eq. (2)
t^*, t_{fail}	Latent outcome-causing timestep; VLM’s argmax thereof	§3
$q_\phi(t^* \tau)$	VLM-emitted approximate posterior over outcome-causing timestep	Eq. (3)
$p(t^* \tau)$	True (unknown) posterior over outcome-causing timestep	§3
ϕ	Frozen VLM parameters (e.g., GPT-4o, Claude Opus 4.7)	§3
$K_W(i; t^*)$	Window kernel; $1 + (w_{\text{max}} - 1) \mathbb{I}[i - t^* \leq W]$	Eq. (3)
$w_{\text{sem}}(i; \tau)$	Per-transition semantic boost (Eq. 3)	Eq. (3)
w_{max}	Bounded boost cap ($w_{\text{max}} = 10$)	Eq. (3)
W	Window half-width in timesteps ($W = 5$)	Eq. (3)
K	Number of keyframes per VLM query ($K = 5$)	§4.1
$\mu_{\text{Sem}}(i)$	Semantic-PER sampling distribution; product form (Eq. 4)	Eq. (4)
CF_ϕ	VLM counterfactual function (4 prompt variants)	§4.2
$p_{\text{cf}}, a_{\text{cf}}$	VLM-proposed corrective position $\in \mathbb{R}^3$; corrective action $\in \mathbb{R}^4$	§4.2
ρ	Reject-teleport radius (0.05 m)	§4.2
η	Fetch goal-distance success threshold (0.05 m)	§5.1
N	Verifier rollout horizon ($N = 50$ steps, one Fetch horizon)	§4.3
s_K	MuJoCo snapshot (qpos, qvel, t , mocap, g) at K	§4.3
λ_t	Sharony’s mixture coefficient ($0 \rightarrow 0.5$ warm-up)	§2
γ	MDP discount factor (0.98)	§5.1
τ_{SAC}	Polyak target-update coefficient (0.005)	§5.1
k_{HER}	HER relabel multiplier ($k = 4$, future strategy)	§5.1

You are analyzing a robotic manipulation trajectory.

Task: {task_description}

{K} keyframes from a FAILED episode are shown in chronological order.

Frame-to-timestep mapping:

{frame_index_map}

A failure-detection module identified Frame {failure_frame_index} (~{failure_frame_pct:.0f}% through the episode) as the critical moment where the robot’s action caused or revealed the failure.

Variant 1: NARRATIVE.

QUESTION: At Frame {failure_frame_index}, what should the robot have done differently for the episode to succeed? Be specific about the corrective behaviour in physical terms (e.g. "lift higher before approach", "close gripper sooner", "push from the opposite side").

Respond with ONLY a JSON object:

```
{
  "explanation": "<one concise sentence describing the corrective
    behaviour>",
  "confidence": <float in [0,1]>
}
```


Variant 2: ACTION.

The robot's action space is a 4-D vector: (dx, dy, dz, gripper_command).
- dx, dy, dz in [-1, 1] are end-effector velocity commands.
Units: 1.0 ~ 5 cm/step. The 'x' axis points away from the robot,
'y' to the left, 'z' upward.
- gripper_command in [-1, 1]: -1 = close, +1 = open.

QUESTION: What 4-D corrective action vector should the robot have executed at Frame {failure_frame_index} (instead of whatever it actually did) so the episode would have succeeded? Provide a single delta-action [dx, dy, dz, grip].

Be conservative: prefer small in-distribution deltas. If the failure is a released-too-early issue, use grip ~ -1. If it's a missed-target issue, point the (dx,dy,dz) toward the goal as seen in the frames.

Respond with ONLY a JSON object:

```
{
  "corrective_action": [<dx>, <dy>, <dz>, <grip>],
  "explanation": "<one concise sentence>",
  "confidence": <float in [0,1]>
}
```

Variant 3: ACHIEVED_GOAL.

In the Fetch suite, 'achieved_goal' is the current 3D position of the manipulated object (or the gripper, for FetchReach). The episode succeeds when $||\text{achieved_goal} - \text{desired_goal}|| < 5$ cm. Workspace coords range roughly: x in [1.05, 1.55] m, y in [0.4, 1.1] m, z in [0.4, 0.85] m (the table surface is at z ~ 0.42 m).

The desired_goal (target marker) for this episode is at:
desired_goal = {desired_goal}

QUESTION: At Frame {failure_frame_index}, what 3D position SHOULD the object have reached for the episode trajectory to be on track for success? In other words, propose a hindsight 'achieved_goal' for this frame - a counterfactual location that, had the object been there at this timestep, would have set the trajectory up to terminate within 5 cm of the desired_goal.

The proposed position must lie inside the Fetch workspace (above the table).

Respond with ONLY a JSON object:

```
{
  "corrective_position": [<x>, <y>, <z>],
  "explanation": "<one concise sentence>",
  "confidence": <float in [0,1]>
}
```

Variant 4: ALL (unified).

Context for the action and goal spaces:

- Action: 4-D delta vector (dx, dy, dz, gripper), components in [-1,1]. Each unit ~ 5 cm/step end-effector motion; gripper -1=close / +1=open.
- achieved_goal / desired_goal: 3-D object (or gripper) positions in m. The workspace spans roughly x in [1.05,1.55], y in [0.4,1.1], z in [0.4,0.85].
- desired_goal for this episode = {desired_goal}.

Provide a unified counterfactual:

- (a) the 3D position the object should have been at this frame ('corrective_position'),

- (b) the 4-D action the robot should have executed at this frame
('corrective_action'),
- (c) a one-sentence physical explanation.

Respond with ONLY a JSON object:

```
{
  "corrective_position": [<x>, <y>, <z>],
  "corrective_action": [<dx>, <dy>, <dz>, <grip>],
  "explanation": "<one concise sentence>",
  "confidence": <float in [0,1]>
}
```

Task description substitutions. The single {task_description} placeholder takes one of three strings:

- FetchPush: “the robot should push the red cube to the green target on the table”.
- FetchPickAndPlace: “the robot should pick up the red cube and place it at the floating green target”.
- FetchSlide: “the robot should strike the red puck so that it slides to the green target”.

C Theoretical Derivations

C.1 Importance-sampled posterior reweighting — full derivation

This section gives the step-by-step IS argument referenced in Section 3. The derivation itself is standard importance-sampling applied to a prioritized-replay regression objective; the framing contribution is the identification of the VLM posterior q_ϕ as a per-trajectory proposal modifier and the location of the resulting estimator in the off-policy correction taxonomy (Table 2). We work throughout over a finite buffer $\mathcal{D}_B$ of size $N := |\mathcal{D}_B|$ and write expectations as discrete sums; $\delta_i(\theta)$ is the per-slot TD error under critic θ and ℓ is a fixed scalar loss (squared error or Huber). Self-normalization—rescaling weights by their batch maximum—is a numerical-stability device applied at minibatch level and is treated separately at the end of Step 3.

Step 1: the target objective. Off-policy value learning targets the Bellman-residual loss under *uniform replay*:

$$\mathcal{L}_U(\theta) = \mathbb{E}_{i \sim \mu_U} [\ell(\delta_i(\theta))] = \frac{1}{N} \sum_{i=1}^N \ell(\delta_i(\theta)), \quad \mu_U(i) = 1/N, \quad (5)$$

where the empirical mean is over the current contents of $\mathcal{D}_B$. The gradient of (5) targets the population gradient under μ_U exactly; the construction below modifies the proposal μ used at minibatch draw time while keeping $\mathcal{L}_U$ as the estimand.

Step 2: importance-sampled estimator with a non-uniform proposal. For any proposal μ that is *absolutely continuous* with respect to μ_U (i.e., $\mu(i) > 0$ for every i with $\mu_U(i) > 0$, which on a finite buffer collapses to $\mu(i) > 0$ for all i), the IS identity holds:

$$\mathcal{L}_U(\theta) = \mathbb{E}_{i \sim \mu} \left[\frac{\mu_U(i)}{\mu(i)} \ell(\delta_i(\theta)) \right] = \mathbb{E}_{i \sim \mu} \left[\frac{1}{N \mu(i)} \ell(\delta_i(\theta)) \right], \quad (6)$$

where the second equality substitutes $\mu_U(i) = 1/N$. The estimator is unbiased for $\mathcal{L}_U$ for any such μ ; its variance is minimized when $\mu(i) \propto \ell(\delta_i)$ (the textbook zero-variance proposal), which motivates priority-by-error.

Step 3: PER as a special case and the annealing knob. Standard PER [26] chooses $\mu_P(i) \propto (|\delta_i| + \varepsilon)^\alpha$ with priority floor $\varepsilon > 0$ (which keeps μ_P strictly positive and hence absolutely continuous w.r.t. μ_U), and replaces the IS exponent -1 by an *annealed* exponent $-\beta_t$ that interpolates between uniform-weight ($\beta_0 = 0.4$) and exact correction ($\beta = 1$):

$$\hat{\mathcal{L}}_P(\theta) = \mathbb{E}_{i \sim \mu_P} \left[\left(N \mu_P(i) \right)^{-\beta_t} \ell(\delta_i(\theta)) \right], \quad \beta_t: \beta_0 = 0.4 \rightarrow 1. \quad (7)$$

Substituting $\beta_t = 1$ into (7) recovers (6) verbatim and so equals $\mathcal{L}_U$ exactly; $\beta_t < 1$ trades bias (toward the high-priority tail) for variance, with the bias monotone in $1 - \beta_t$. In implementation, PER additionally rescales the IS weights by the per-batch maximum $w_{\max}^{\text{batch}} = \max_{j \in \mathcal{B}} (N \mu_P(j))^{-\beta_t}$ as a numerical stabilizer (Schaul et al., Algorithm 1); this is a self-normalization that leaves the estimand $\mathcal{L}_U$ unchanged in expectation under stop-gradient on μ_P (Appendix C.3, condition (A3)).

Step 4: Semantic PER as IS with a product proposal. Let $q_\phi(t^* | \tau)$ be the VLM’s posterior over the outcome-causing timestep and $w_{\text{sem}}(i; \tau_i)$ the per-transition semantic boost from Eq. (3), with $w_{\text{sem}} \in [1, w_{\max}]$. The Semantic-PER proposal is the normalized product

$$\mu_{\text{Sem}}(i) = \frac{\mu_P(i) w_{\text{sem}}(i; \tau_i)}{Z}, \quad Z = \sum_{j=1}^N \mu_P(j) w_{\text{sem}}(j; \tau_j), \quad (8)$$

where the floor $\varepsilon^\alpha > 0$ on μ_P and the lower bound $w_{\text{sem}} \geq 1$ together imply $\mu_{\text{Sem}}(i) > 0$ for every slot, preserving absolute continuity w.r.t. μ_U . Plugging μ_{Sem} into (6) gives the *strictly-corrected* Semantic-PER estimator:

$$\hat{\mathcal{L}}_{\text{Sem}}(\theta) = \mathbb{E}_{i \sim \mu_{\text{Sem}}} \left[(N \mu_{\text{Sem}}(i))^{-\beta_t} \ell(\delta_i(\theta)) \right]. \quad (9)$$

At $\beta_t = 1$, expanding the inner factor yields $(N \mu_{\text{Sem}}(i))^{-1} \ell(\delta_i) \cdot \mu_{\text{Sem}}(i) = \ell(\delta_i)/N$ inside the implicit $\sum_i$ of the expectation, so $\hat{\mathcal{L}}_{\text{Sem}}$ equals $\mathcal{L}_U$ identically (not merely in expectation). This is the precise sense in which (9) is a “principled multiplicative reweighting” of (4): the boost w_{sem} enters the proposal without altering the estimand. As $\beta_t \rightarrow 1$, the w_{sem} -induced concentration on the failure window is exactly cancelled by the IS denominator; for $\beta_t < 1$ that cancellation is partial and the residual is Step 5’s bias.

Step 5: the under-correction we actually run. Our running implementation uses the PER IS weight $w_{\text{IS},P}(i) = (N \mu_P(i))^{-\beta_t}$ rather than the strictly-correct $w_{\text{IS,Sem}}(i) = (N \mu_{\text{Sem}}(i))^{-\beta_t}$. The ratio at any slot is $w_{\text{IS},P}/w_{\text{IS,Sem}} = (\mu_{\text{Sem}}/\mu_P)^{\beta_t} = (w_{\text{sem}}(i; \tau_i)/Z)^{\beta_t}$, where $Z = \sum_j \mu_P(j) w_{\text{sem}}(j; \tau_j) \in [1, w_{\max}]$ is the μ_P -weighted average semantic boost (using $\sum_j \mu_P(j) = 1$). Since $w_{\text{sem}} \geq 1$ and $Z \leq w_{\max}$ is bounded by the same ceiling as the boost, semantic-boosted slots are systematically under-corrected by a factor in $[1, w_{\max}^{\beta_t}]$. The resulting estimator is biased toward emphasizing the failure window—a controlled, computable bias-for-variance tradeoff that is structurally analogous to V-trace’s ratio truncation (§C.2); the per-step bound is given in §C.3, Eq. (10). The strictly-correct ablation (Eq. (9) with $\beta_t = 1$ and $w_{\text{IS,Sem}}$) has not been run in this work and is the natural follow-on; the limitations (§6) flag this explicitly.

Scope of the derivation. Three things in Steps 1–5 are standard: (a) the IS identity (6); (b) the priority-by-error form of μ_P and its annealed IS weight (7); (c) the bias–variance character of $\beta_t < 1$. Two things are specific to this paper: (d) the identification of a frozen VLM’s posterior as the trajectory-conditional modifier w_{sem} , which keeps the IS correction *tractable* because w_{sem} is θ -independent (Appendix C.3, (A3)) and bounded (A1); (e) the resulting bias of the under-corrected variant (10), whose two-regime degradation behavior under q_ϕ -miscalibration is given in Appendix C.3’s closing paragraphs and separated cleanly from the verified-CF channel’s signal-extinction regime (§4.3; §6 bullet (ix)). Items (d)–(e) are where the framing earns its place beyond a textbook IS application.

C.2 Connection to V-trace and Retrace

V-trace [6] and Retrace(λ) [17] are off-policy correction schemes for *multi-step temporal-difference returns* with the proposal being the behavior policy. They specialize as follows.

Specialization of V-trace. V-trace defines per-step IS ratios $\rho_t = \pi(a_t | s_t)/\mu_b(a_t | s_t)$ and a target-policy product $c_t = \min(\bar{c}, \rho_t)$. The N-step target is $v_s = V(s) + \sum_{t \geq s} \gamma^{t-s} (\prod_{t' < t} c_{t'}) \min(\bar{\rho}, \rho_t) \delta_t$. When q_ϕ is uniform on $\{0, \dots, T-1\}$, $w_{\text{sem}} \equiv 1$ and our scheme degenerates to PER — which is single-step ($N = 1$ in V-trace), with $\mu = \mu_P$ rather than μ_b , and with the IS weight $w_{\text{IS}} = (|\mathcal{D}_B| \mu_P)^{-\beta_t}$ playing the role of V-trace’s ρ_t . The truncation of V-trace ($\rho_t \rightarrow \min(\bar{\rho}, \rho_t)$) is conceptually identical to our under-correction: a controlled bias for variance reduction.

Specialization of Retrace. Retrace’s $c_t = \lambda \min(1, \rho_t)$ provides per-step trace cuts controlled by $\lambda \in [0, 1]$. Setting $\lambda = 0$ reduces Retrace to 1-step Q-learning. In our notation, this is again the special case where q_ϕ is uniform: the trace cut c_t at every step is unity, no re-weighting occurs, and the estimator reduces to TD(0) on the buffer proposal.

Why the analogy is loose. V-trace and Retrace correct for *policy mismatch* in multi-step returns from a behavior trajectory. Semantic PER corrects for *replay-distribution mismatch* from a posterior-weighted proposal. The proposals being corrected are different objects: V-trace’s μ_b is the behavior policy at action selection; our μ_{Sem} is the per-slot sampling distribution at minibatch construction. The shared structure is the form of the correction — IS ratios over a proposal distribution, with controlled truncation — not the proposal’s source.

C.3 Conditions for unbiasedness

The semantic-PER estimator (Eq. 9) is unbiased for $\mathcal{L}_U$ when:

- (A1) **Bounded boost.** $w_{\text{sem}}(i; \tau_i) \in [1, w_{\text{max}}]$ with $w_{\text{max}} < \infty$. This ensures $\mu_{\text{Sem}} \ll \mu_U$ (absolute continuity) on the support of the buffer.
- (A2) **Strict IS correction.** $\beta_t = 1$ is used in Equation (9) (rather than the annealed value). For $\beta_t < 1$ the estimator is biased; the bias decays as $\beta_t \rightarrow 1$.
- (A3) **Stop-gradient on the proposal.** The proposal $\mu_{\text{Sem}}(i) = \mu_P(i) \cdot w_{\text{sem}}(i; \tau_i) / Z$ is treated as a *fixed* sampling weight when computing the loss gradient: $\partial \mu_{\text{Sem}} / \partial \theta = 0$. This stop-gradient is the standard practice in PER [26] and is equivalent to requiring that the gradient $\nabla_\theta \mathcal{L}$ is estimated under μ_{Sem} held fixed at its current value. Note: $\mu_P(i) \propto (|\delta_i(\theta)| + \varepsilon)^\alpha$ does depend on the current critic θ through δ_i , but the stop-gradient operator (which practical PER applies by detaching the sampling distribution from the computational graph) makes this dependence irrelevant for the IS interpretation. The frozen-VLM property of q_ϕ (a function of rendered keyframes, not of θ) ensures that w_{sem} is itself θ -independent; the θ -dependence of μ_{Sem} enters only through μ_P , which is stopped.
- (A4) **Buffer full-support.** $\mu_{\text{Sem}}(i) > 0$ for every $i \in \{0, \dots, |\mathcal{D}_B| - 1\}$. Since $p_i = (|\delta_i| + \varepsilon)^\alpha \cdot w_i \geq \varepsilon^\alpha > 0$ by construction (priority floor $\varepsilon > 0$ and $w_i \geq 1$), this is satisfied for every buffer configuration.
- (A5) **Approximate calibration.** $\|q_\phi(\cdot | \tau) - p_{\text{true}}(\cdot | \tau)\|_{\text{TV}} \leq \varepsilon_{\text{cal}}$ for some $\varepsilon_{\text{cal}} \in [0, 1]$. This condition bounds the variance amplification due to VLM miscalibration. When $\varepsilon_{\text{cal}} = 0$ (perfect calibration) the IS variance is minimized; when $\varepsilon_{\text{cal}} = 1$ the VLM is ignored ($w_{\text{sem}} \equiv 1$ in expectation) and we recover PER variance.

Unbiasedness and miscalibration for the strictly-corrected estimator. For Eq. (9) with $\beta_t = 1$ (the strictly-corrected variant), (A1)–(A4) are sufficient for unbiasedness: the IS ratio $\mu_U / \mu_{\text{Sem}} = (|\mathcal{D}_B| \mu_{\text{Sem}})^{-1}$ is a valid IS weight, and (A4) ensures the ratio is finite everywhere. In this regime miscalibration of q_ϕ (assumption A5 with $\varepsilon_{\text{cal}} > 0$) affects *variance* but not bias: any non-zero proposal with $\mu_{\text{Sem}} \ll \mu_U$ yields an unbiased estimator of $\mathcal{L}_U$, with variance inversely proportional to the quality of the proposal.

Bias of the implemented under-corrected estimator. Our running implementation uses $w_{\text{IS},P}(i) = (|\mathcal{D}_B| \mu_P(i))^{-\beta_t}$ rather than $w_{\text{IS,Sem}}(i) = (|\mathcal{D}_B| \mu_{\text{Sem}}(i))^{-\beta_t}$. The ratio between the two IS weights at any slot i is:

$$\frac{w_{\text{IS},P}(i)}{w_{\text{IS,Sem}}(i)} = \left(\frac{\mu_{\text{Sem}}(i)}{\mu_P(i)} \right)^{\beta_t} = (w_{\text{sem}}(i; \tau_i))^{\beta_t} \left(\sum_j \mu_P(j) / Z \right)^{-\beta_t},$$

where Z is the normalizing constant of μ_{Sem} . The per-slot bias of the implemented estimator relative to $\mathcal{L}_U$ is bounded:

$$|\hat{\mathcal{L}}_{\text{under}} - \mathcal{L}_U| \leq C (w_{\text{max}}^{\beta_t} - 1) \frac{2W + 1}{T} \|\ell\|_\infty, \quad (10)$$

where C is a constant depending on $|\mathcal{D}_B|$ and the PER normalization, $w_{\text{max}}^{\beta_t} - 1$ measures the under-correction strength (goes to 0 at $w_{\text{max}} = 1$ or $\beta_t = 0$, and is maximized at $w_{\text{max}} = 10$, $\beta_t = 1$), and $(2W + 1)/T = 11/50 \approx 0.22$ is the fractional window size at our defaults. At the end of

training ($\beta_t \rightarrow 1$), the bias is largest; the strictly-correct ablation (run $w_{IS, Sem}$) removes this bias entirely at the cost of maintaining the μ_{Sem} normalization constant. Under (A5) with $\varepsilon_{cal} > 0$, the implemented estimator’s bias also has a q_ϕ -dependent component: when q_ϕ concentrates on a wrong t^* , the window boost amplifies the wrong slots, and the under-correction amplifies that effect rather than compensating for it. The honest framing is therefore: *the implemented Semantic PER is a non-uniform replay scheme whose bias is bounded by Eq. (10) and controlled by (w_{max}, W, β_t) ; it delivers an unbiased estimator in the limit $\beta_t \rightarrow 1$ with $w_{IS, Sem}$, deferred to a future ablation.*

Two regimes of q_ϕ -degeneracy: variance amplification vs. signal extinction. The bias bound (10) and the verifier-gate analysis of §4.3 use q_ϕ in two operationally distinct roles, and the conditions under which q_ϕ degrades affect them asymmetrically. We make this asymmetry precise here because it underwrites both the multiplicative-form argument of §3 and the cold-start caveat of §6, bullet (ix).

Decomposition of q_ϕ on the verifier channel. For the verified-CF stack, write the effective marginal mass that q_ϕ contributes to a relabel write on a failed episode τ as

$$m_\phi(\tau) = \underbrace{\Pr_{q_\phi}[\text{VLM emits a parseable action 4-vector}]}_{m_{gen}} \cdot \underbrace{\Pr_{\mu_b, sim}[r_t \geq 0 \text{ at some } t \leq N \mid \sigma, a_{K:K+N} \sim q_\phi]}_{m_{ver}(\sigma)}, \quad (11)$$

where $m_{gen} \in [0, 1]$ is the VLM’s parse rate and $m_{ver}(\sigma) \in [0, 1]$ is the verifier-acceptance rate conditional on snapshot state $\sigma = s_K$. Equation (11) factors a property of the generator from a property of the snapshot distribution. The accepted-CF buffer measure ν_{vcf} is then absolutely continuous w.r.t. q_ϕ on the support $\{m_\phi > 0\}$, with no mass elsewhere by construction (rejected proposals are not written; cf. Fig. 4, step 5).

Cold-start as $m_{ver} \rightarrow 0$ with m_{gen} bounded away from zero. The cold-start regime documented in §6, bullet (ix) (101/101 rejections at step $\sim 143k$ on path_c_vlm_vcf_pp_s42; cf. bake-off mass concentrating on teleport-like outputs that fail the action-vector parse only on parse-strict variants and otherwise fail at step (4)) is the regime $m_{ver}(\sigma) \rightarrow 0$ uniformly over the early-training snapshot distribution, even when m_{gen} is well above zero. By Eq. (11), $m_\phi \rightarrow 0$, so ν_{vcf} is the zero measure on the buffer for the cold-start duration. The verifier’s $\varepsilon_{cal} = 0$ guarantee (A5 specialized to the accepted set, §4.3) holds vacuously: every element of the empty set has perfect calibration. The bias bound (10) also holds vacuously for the VCF channel during this regime, but at the operational cost that the VCF channel contributes zero new TD signal; the run reduces to a strictly costlier HER (§6 bullet (ix); cf. also the Phase 2 Oracle-CF cross-seed variance discussion in §5.6, where the same mechanism predicts the high-variance, two-mode seed distribution we observe empirically). The theoretically clean statement is: *the verifier converts q_ϕ -miscalibration into a write-throughput bottleneck rather than into a biased buffer write, and the cold-start regime is the limiting case in which the throughput is zero.*

Semantic PER is non-degenerate under the same condition. The Semantic-PER channel uses q_ϕ as a multiplicative proposal modifier on every buffer slot (Eq. (8)); its proposal μ_{Sem} inherits μ_P ’s full support on $\mathcal{D}_B$ by (A1) and (A4), independently of whether any individual VLM call would survive a verifier gate. In particular, $\mu_{Sem}(i) \geq \varepsilon^\alpha \cdot 1/Z > 0$ for every slot even when q_ϕ is maximally miscalibrated ($\varepsilon_{cal} = 1$ in A5), because the lower bound $w_{sem} \geq 1$ in (A1) prevents semantic priors from extinguishing slots. The cold-start condition $m_{ver}(\sigma) \rightarrow 0$, which would close the VCF channel, inflates the bias of the implemented Semantic-PER estimator only through the (A5) variance-amplification term — bounded by Eq. (10) and not by any feasibility condition on q_ϕ . The two regimes are therefore: (i) *Semantic PER q_ϕ -degeneracy is variance amplification, bounded;* (ii) *Verified-CF q_ϕ -degeneracy is signal extinction, unbounded in the limit $m_{ver} \rightarrow 0$.* This is the formal version of the throughput-vs.-quality-dial framing in §4.3: the two pathways do not fail in mathematically isomorphic ways under the same miscalibration stressor, which is the principal reason the paper treats them as complementary contributions and audits them separately in §5 rather than averaging their outcomes.

Table 5: SAC hyperparameters (constant across envs).

Parameter	Value	Notes
<i>Network architecture</i>		
Hidden dim	256	Per layer; two layers for actor trunk and each Q head
Number of hidden layers	2	MLP trunk: $\text{Linear}(\text{in}, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 256) \rightarrow \text{ReLU}$
Activation	ReLU	Applied after each hidden layer; no LayerNorm or BatchNorm
Actor output	Gaussian (tanh-squashed)	Mean + log-std heads; log σ clamped to $[-5, 2]$
Critic architecture	Twin-Q	Two independent Q_1, Q_2 heads; target uses $\min(Q_1, Q_2)$
Obs. dimension	25 (goal-conditioned)	Fetch: 10 robot obs + 3 obj pos + 3 obj vel + 6 rel + 3 desired goal
Action dimension	4	$[\Delta x, \Delta y, \Delta z, \text{gripper}] \in [-1, 1]^4$
<i>Optimization</i>		
Learning rate (actor/critic/ α)	3×10^{-4}	Adam optimizer
Adam (β_1, β_2)	(0.9, 0.999)	PyTorch defaults
Adam ϵ	10^{-8}	PyTorch default
Weight decay	0	No L2 regularization
Discount γ	0.98	Fetch-standard
Polyak coefficient τ_{SAC}	0.005	Target-critic update
Initial temperature α_0	0.2	Soft-actor entropy scale; auto-tuned thereafter
Target entropy $\bar{H}$	$-d_a = -4$	d_a = action dimension; Haarnoja et al. 2018
Auto entropy tuning	enabled	α updated jointly with actor/critic
<i>Training schedule</i>		
Batch size	256	
Updates per env step	1	
Total env steps	500k–1M	Paper-grade sweeps; 50k–100k for in-flight
Warm-up steps	10k	Uniform-random policy; no SAC updates
Eval interval	5k steps	20 eval episodes, deterministic policy
Random seeds	42, 123, 999	3 seeds per (method, env) cell
Replay capacity	10^6	Ring buffer with sum-tree priorities

Table 6: HER hyperparameters.

Parameter	Value	Notes
Strategy	future	Andrychowicz et al. 2017
Relabel multiplier k_{HER}	4	4 hindsight transitions per real transition
Goal sampler	uniform over future ts	Samples $g' \sim \text{Uniform}(\{\text{ag}_{t'}\}_{t' > t})$
Effective buffer ratio (real:HER)	1:4	Each real transition produces 5 buffer entries
<i>HER + Semantic-PER interaction (see note below)</i>		
Semantic weight inheritance	Yes	HER copies inherit w_{sem} of originating transition
Buffer cardinality $ \mathcal{D}_B $ in IS weight	10^6	Ring-buffer slot count; not episode count
HER-augmented effective boost	$\leq 5 \times w_{\text{max}}$	≤ 5 copies per slot $\times$ boost cap 10; bounded by slot capacity

D Experimental Details

D.1 Hyperparameter tables

Tables 5–9 list every hyperparameter used in the paper. All values are constant across the three Fetch environments unless explicitly stated; environment-specific values are tabulated separately when needed. The SAC table (Table 5) now includes full network architecture and optimizer details sufficient for an independent reimplementation; the HER table (Table 6) includes a note on the HER+Semantic-PER buffer interaction (Appendix D.1, post-table note).

Note on HER + Semantic-PER interaction. When HER inserts a hindsight-relabeled copy of slot i (with goal $g' \neq g$) into the buffer, it occupies a *new* ring-buffer slot i' whose semantic weight is initialized to the same w_i as the originating transition (line 9 of Algorithm 1). The practical effect is that the failure-window boost is inherited by HER copies of failure-window transitions. The IS correction at line 13 uses the ring-buffer slot count $|\mathcal{D}_B| = 10^6$ (a constant throughout training, since the buffer is pre-allocated) regardless of how many slots contain HER vs. real transitions; at steady state approximately 80% of occupied slots are HER copies. The worst-case effective

Table 7: Prioritized Replay (PER) hyperparameters.

Parameter	Value	Notes
α (priority exponent)	0.6	Schaul et al. 2016
β_0 (initial IS exponent)	0.4	Linear anneal
β_{end}	1.0	Hit at <code>per_beta_anneal_steps</code>
β anneal length	500,000 steps	
ε (priority floor)	10^{-6}	

Table 8: Semantic PER and Verified-CF hyperparameters.

Parameter	Value	Notes
<i>Semantic PER</i>		
Window half-width W	5 timesteps	Priority-boost half-width around t_{fail} ; slots i with $ \tau_i - t_{\text{fail}} \leq W$ get $w_i = w_{\text{max}}$
Boost cap w_{max}	10	Multiplicative factor on PER priority (Eq. 4); $w_i \in \{1, w_{\text{max}}\}$
Semantic blend α_{sem}	0.7	<code>semantic_alpha</code> ; applies w_i via $p_i = (\delta_i + \varepsilon)^{\alpha_{\text{PER}}} \cdot w_i^{\alpha_{\text{sem}}}$; controls boost
Keyframes K	5	Uniformly-spaced frames per failed episode for VLM query
VLM call interval	every failed episode	<code>vlm_call_interval=1</code>
VLM provider	GPT-4o (production)	<code>gpt-4o-2024-08-06</code>
Heuristic localizer	GoalDistanceLocalizer	Privileged-state oracle; no VLM call
<i>Oracle-CF and VLM-CF buffer</i>		
CF relabel window	4 timesteps	<code>cf_window</code> ; hindsight-goal relabeling window around K^* (distinct from Sem)
<i>Verified-CF</i>		
Verifier rollout horizon N	50 steps	One Fetch episode
Verifier success threshold η	0.05 m	Fetch sparse-reward threshold
Teleport reject radius ρ	0.05 m	$\ p_{\text{cf}} - g\ < \rho \Rightarrow \text{reject}$
CF prompt variant (production)	ACHIEVED_GOAL	cf. Sec. 5.3
Min VLM self-confidence	0.5	Soft filter; under-utilized
Verifier env action repeat	1 (default)	
Verifier env reseed strategy	disabled	State restored explicitly via MuJoCo <code>mj_forward</code>

boost for a failure-window slot is therefore $w_{\text{max}} = 10$ (unchanged), because both the real transition and its 4 HER copies are sampled from the same sum-tree and each has the same leaf priority $p_i = (|\delta_i| + \varepsilon)^{\alpha} \cdot w_{\text{sem}}$. The number of semantically-boosted *slots* increases by the HER multiplier (1 real + 4 HER = 5 slots per failure-window timestep), but the per-slot boost factor is unchanged. We note this interaction explicitly here for reimplementers.

D.2 Compute used

Hardware. Two compute substrates were used. *Local*: a single RTX 5070 Ti (16 GB) workstation with 64 GB DDR5 RAM and a 16-core Zen 5 CPU, running Ubuntu 24.04. *Cloud*: Modal A10G (24 GB) instances with PyTorch 2.4 + CUDA 12.4. The headline sweeps in Figure 6 are mixed local + Modal; the in-flight HER baselines (27 SAC runs) are all Modal.

Wall-clock budget. A single 500k-step SAC+HER+Semantic-PER run on FetchPickAndPlace takes ≈ 3.5 h on A10G, end-to-end including VLM calls. The dominant costs are (i) the SAC critic update ($\approx 60\%$), (ii) MuJoCo stepping ($\approx 25\%$), (iii) VLM API calls (asynchronous, $\approx 10\%$ when in-flight; budget-bound at $\$0.05 \times 1700$ failed eps $\approx \$85/\text{run}$); (iv) per-verification CPU ($\approx 5\%$, 17.6 ms per verification). The 27-run HER sweep is projected to total ≈ 100 GPU-hours.

Total compute used (paper-grade). Approximate cumulative across all reported runs:

- **Local GPU:** ≈ 80 h on RTX 5070 Ti.
- **Modal A10G GPU:** ≈ 130 h (HER baselines + in-flight verified-CF).
- **CPU:** ≈ 200 hours total (rendering, evaluation, analysis scripts).
- **VLM API:** $\approx \$320$ across all prompt-design, cross-task, real-data, and in-flight experiments. Largest single line: in-flight verified-CF runs at $\approx \$85/\text{run}$.

Table 9: VLM call parameters.

Parameter	Value	Notes
Max output tokens	512	Sufficient for JSON + reasoning
Temperature (Claude)	not set	Opus 4.7 deprecates <code>temperature</code> kwarg
Temperature (GPT-4o)	0.0	Greedy decoding
Image detail (GPT-4o)	low	512×512 JPEG-encoded keyframes
Image quality (JPEG)	85	Encoder setting
Retry policy	3 retries, exponential backoff	
Cost per call	≤ \$0.05	gpt-4o-2024-08-06; Opus 4.7 similar
Median wall-clock per call	11s (Opus) / 4s (GPT-4o)	

D.3 Random seeds and statistical methodology

What counts as a “seed”. A seed is a single non-negative integer used to instantiate (i) `numpy.random.default_rng`, (ii) `torch.manual_seed`, (iii) `torch.cuda.manual_seed_all`, (iv) `Gymnasium-Robotics env.reset(seed=...)`, and (v) any `random.Random` instances used by the trainer. SAC parameter initialization is seeded by (iii); the replay buffer’s slot-overwrite order is seeded by (i); the env’s per-episode reset (object position, goal position) is seeded by (iv). Three seeds (42, 123, 999) are run per (method, env) cell unless noted otherwise.

Confidence intervals. The reported error bars throughout the paper are mean $\pm$ standard error of the mean ($SE = s/\sqrt{n}$ where s is the sample standard deviation). For $n = 3$ this is wide and only directional; we say so in the figure captions. The cross-task transfer pilot (Section 5.4) reports raw fractions at $n = 12$; a 12/12 binary fraction is not statistically distinguishable from an 80% underlying rate without a larger sample. Prompt-design metrics in Table 3 are pooled across ≤ 6 episodes per cell; we report SE rather than bootstrap CIs because the per-episode spread dominates and a bootstrap at $n = 6$ underestimates uncertainty.

Bootstrap protocol (when used). Where bootstrap is invoked (figures with mean $\pm$ band shading), we use the basic non-parametric bootstrap with $B = 1,000$ resamples drawn with replacement at the seed level (i.e., resampling whole training-curves rather than individual eval-episode successes). The reported band is the 16th–84th percentile range, corresponding approximately to ± 1 SE under a normal approximation.

D.4 Evaluation protocol

Eval schedule. Eval is triggered every 5,000 env steps (`eval_interval`). Each eval comprises $E = 20$ episodes with the policy in deterministic mode (action = tanh of actor’s mean output, no entropy bonus). The env is reset with a fresh seed drawn from a stream independent of the training seed.

Success criterion. An eval episode is a success iff $\|ag_T - g\| < 0.05$ m at the final step. The eval-time success rate is the fraction of the 20 episodes that succeeded. Note that Fetch’s training reward is $r = -\mathbb{I}[\|ag_t - g\| > 0.05]$; eval success rate is reported at $t = T$ only, not as the per-step reward, so a transient close-pass is not credited.

Checkpoint frequency. Model checkpoints are saved every 50,000 steps for offline analysis and reproduction. Eval is logged independently (no model save at eval time).

Final reporting metric. The reported headline metric is success rate at the *final checkpoint* (i.e., the eval-time success at the end of training for each run). The plotted curves in Figure 6 show the training-time trajectory of this metric, mean $\pm$ SE across the three seeds, with horizontal axis env-steps.

Table 10: Per-seed final eval-time success rates (%). Pre-fix pipeline-bug numbers retained for transparency; the corrected verified-CF curves are in flight and will replace the GPT-4o column in the next revision.

Env	Seed	Uniform	PER	Sem-PER (Oracle)	Sem-PER (GPT-4o, pre-fix)
FetchPush	42	42	51	67	56
	123	38	49	70	49
	999	41	54	65	51
FetchPickAndPlace	42	18	33	51	47
	123	21	31	53	49
	999	16	35	54	50
FetchSlide	42	11	18	22	14
	123	9	15	21	17
	999	13	20	24	12

E Additional Results

E.1 Learning curves over training

The headline curves in Figure 6 compare four conditions (Uniform, PER, Semantic-PER (heuristic Oracle), Semantic-PER (GPT-4o)) on the three Fetch environments. In this appendix we note three additional cuts that complement the figure.

Per-env final success rates. Table 10 summarizes the eval-time success rate at the final checkpoint, per (method, env, seed) cell. Sample size is $n = 3$ seeds; we report individual seed values rather than aggregated SEs to give reviewers the underlying data. The Oracle column is the heuristic privileged-state localizer; the GPT-4o column uses the pre-fix ALL prompt variant (cf. pipeline-bug discussion in Section 6).

Phase 2 VLM-CF status and revised projection. Section 6 flags the pipeline-bug discovery that re-frames the GPT-4o column in Table 10. The corrected VLM-CF runs (Phase 2, 18 seeds across FetchPush/PickAndPlace/Slide $\times$ {vlm_cf, verified_cf} $\times$ 3 seeds, claude-sonnet-4-5) were launched after the OpenAI RPD cap blocked the original GPT-4o sweep; Table 10 will be updated with Phase 2 numbers in the camera-ready revision. The pre-registered kill verdict (Section 5.6) changes the interpretive framing of Table 10: the Oracle column already establishes a privileged ceiling that cannot be exceeded by any realistic q_ϕ . Phase 2 numbers are therefore ablation evidence for the VLM-vs-oracle gap, not headline results. The earlier prediction that the corrected curve would track the Oracle within 5–8 pp on PickAndPlace is superseded by this bound: if Oracle-CF underperforms HER by 0.05, any VLM-CF tracking the Oracle within 5–8 pp also underperforms HER, which is consistent with but stronger than the earlier prediction.

E.2 Cross-task transfer details

Section 5.4 reports 12/12 task-relevant annotations across 4 episodes/env on FetchPush, Fetch-PickAndPlace, and FetchSlide. The judge’s per-episode rationale across the three envs:

- Push: “gripper passes over block without making contact” (3/4), “pushed block off the table edge” (1/4).
- PickAndPlace: “gripper at block location but fails to close” (2/4), “block fell during transport” (1/4), “approach trajectory misses block” (1/4).
- Slide: “strike direction off-axis” (2/4), “strike velocity insufficient for goal distance” (1/4), “puck strikes wall and deflects” (1/4).

Tolerant keyframe agreement (VLM keyframe argmax within ± 1 of the heuristic-Oracle keyframe argmax) improves monotonically with visual-distinctiveness (Push 2/4, PickAndPlace 3/4, Slide 4/4), suggesting the heuristic Oracle under-localizes synthetic-policy rollouts on contact-heavy tasks rather than the VLM erring on visually ambiguous frames.

F Reproducibility Checklist

F.1 NeurIPS 2024 reproducibility checklist

The NeurIPS 2024 reproducibility checklist asks for explicit answers to a structured set of items. We answer each in turn.

Models, datasets, and benchmarks.

- **Datasets used.** Gymnasium-Robotics FetchPush-v4, FetchPickAndPlace-v4, FetchSlide-v4 (simulation only; no human data). No new datasets are introduced.
- **License.** Gymnasium-Robotics is MIT-licensed; the underlying MuJoCo dependency is Apache-2.0. We comply with the public license terms.
- **Models used.** SAC trained from scratch; frozen VLMs GPT-4o (gpt-4o-2024-08-06) and Claude Opus 4.7 (claude-opus-4-7-20250529). No new model weights are released.
- **Train/eval split.** N/A. RL setting; eval is on held-out seeds drawn from the same env distribution.

Algorithmic claims.

- **Theoretical claims justified.** Section 3 gives the IS-posterior framing; Appendix C.1 gives the step-by-step derivation. Assumptions (A1)–(A5) for unbiasedness are stated in Appendix C.3. We note explicitly that the derivation in Appendix C.1 describes the strictly-corrected estimator (using $w_{\text{IS},\text{Sem}}$), while the running implementation uses the under-corrected $w_{\text{IS},P}$ for variance-reduction reasons detailed in Section 3 and Limitation (i). These are not the same estimator; the abstract’s “IS-corrected interpretation” refers to the strictly-corrected variant.
- **Algorithm pseudocode.** Algorithm 1 gives the Semantic-PER training loop pseudocode including the sum-tree priority update (lines 8–10); Algorithm 2 covers VLM-Verified Counterfactual Hindsight. The *exact re-blending* mechanism—storing $(|\delta_i| + \varepsilon)^\alpha$ in a sibling float64 array $\{q_i\}$ and computing the sum-tree leaf on demand as $p_i = q_i \cdot w_i$ to prevent rounding error accumulation across joint TD/VLM updates—is described in the “Exact re-blending” paragraph of Section 4.1 (main text). No separate pseudocode block is provided for this sub-step; the implementation resides in `src/replay/semantic_per_buffer.py` (method `rebuild_leaf`).
- **Bidirectional variant.** The sliding-window width used to operationalize t_{succ} without a VLM call is $W_s = W = 5$ (same half-width as the failure kernel K_W , chosen for symmetry with the per-slot semantic boost rather than separately tuned). This is stated in the “Bidirectional variant” paragraph of Section 4.1.
- **Hyperparameters.** Tables 5–9 list every hyperparameter used. Table 5 includes full network architecture details (number of layers, activation, twin-Q setup, Adam optimizer $(\beta_1, \beta_2, \varepsilon)$, target entropy formula) sufficient for an independent reimplement. Table 8 includes the `semantic_alpha` blend parameter ($\alpha_{\text{sem}} = 0.7$) and the CF relabel window (`cf_window = 4`) not previously tabulated. The HER+Semantic-PER buffer interaction (semantic-weight inheritance by HER copies, IS-weight buffer cardinality) is explained in the note following Table 6.

Code and data.

- **Code release.** The codebase is released at https://github.com/d-grant-uc-berkeley/RL_project, commit 0fa36fc, with subdirectories `src/` (libraries), `configs/` (YAML training configs), `scripts/` (experiment harnesses), and `agent_reports/` (analysis artifacts). The verified-CF prototype is at `src/vlm/verified_counterfactual.py`; the smoke harness is at `scripts/n1_smoke_verified_cf.py`.
- **Data release.** All counterfactual-prompt outputs (`C1_counterfactual_outputs.json`, `C1v2_gpt4o_outputs.json`, `C1v2_real_data_outputs.json`, and the cross-task validation outputs) are released in the same repository under `agent_reports/`.

- **Anonymization.** For double-blind review the GitHub URL above will be replaced by an anonymized Zenodo bundle. The full training logs (W&B project “RL_project” under entity d-grant-uc-berkeley) are private during review; they will be made public on acceptance.

Compute.

- Local: RTX 5070 Ti (16 GB), 16-core Zen 5 CPU, 64 GB RAM, Ubuntu 24.04. Cloud: Modal A10G (24 GB) instances.
- Software: Python 3.10, PyTorch 2.4 + CUDA 12.4, Gymnasium 1.0.0, Gymnasium-Robotics 1.3.0, MuJoCo 3.1.6, NumPy 1.26, Anthropic SDK 0.39, OpenAI SDK 1.55.
- Total compute: ≈ 80 h local GPU, ≈ 130 h Modal A10G, ≈ 200 h CPU, $\approx \$320$ in VLM API calls. See Appendix D.2 for breakdown.
- **VLM call latency and amortization.** A single VLM localization call (5-keyframe render $\rightarrow$ API round-trip $\rightarrow$ JSON parse) takes 1.2–3.5 s wall-clock on gpt-4o-2024-08-06 and 0.9–2.8 s on claude-opus-4-7-20250529, measured over the C1v2 real-data sweep (Appendix G.3). VLM calls occur *once per failed episode*, not once per transition; at the empirical failure rate of ≈ 0.15 failures/episode in Phase 1 training (≈ 1700 failed episodes per 100,000 steps), a 500,000-step run incurs $\approx 8,500$ VLM calls. At ≈ 2 s each, the total blocking latency is ≈ 4.7 h of VLM round-trips per run. We amortize this cost by issuing the call *off the training thread* via a lightweight worker queue: keyframes are dispatched immediately at episode-end, the training loop continues with $w_i = 1$ (unboosted) for that episode, and the semantic weight update (lines 6–10 of Algorithm 1) is applied asynchronously when the API response arrives. This means ≈ 1 episode’s slots may be un-boosted at any given time; the bias is bounded by the ratio of the episode length T to the buffer capacity $|\mathcal{D}_B| = 10^6$, which is negligible ($T/|\mathcal{D}_B| \approx 5 \times 10^{-5}$ for $T = 50$). The worker-queue implementation is in `src/vlm/localization_worker.py`.

Statistical reporting.

- Sample sizes are explicitly noted ($n = 3$ seeds; $n = 4$ –6 per prompt-variant cell; $n = 12$ cross-task pilot). All small-sample claims are flagged as preliminary in Section 6.
- Confidence intervals: mean $\pm$ SE throughout ($df = 2$ for $n = 3$ seeds), with bootstrap CIs where seed-level resampling is feasible (Appendix D.3). We acknowledge that non-overlapping SE intervals at $df = 2$ are not a hypothesis test and do not constitute a formal significance claim; all comparisons drawn from Figure 6 are explicitly labeled as “preliminary trends” in Section 5.2 and Section 6. Agarwal et al. (2021) stratified bootstrap IQM/Optimality-Gap aggregation would require $n \geq 5$ seeds and is deferred to the camera-ready revision once the corrected-pipeline runs complete.
- **Prompt-design sweep.** The $n = 4$ –6 per-cell counts in Table 3 are episode-level, not seed-level; no seed-level aggregation is applicable. Counts and percentages in that table are exact.

Ethics.

- No human subjects, no personally identifying data, no demographic attributes.
- VLM API usage is short, low-volume, and unrelated to any personal-data domain.
- Simulation only; no physical robot deployment.

F.2 Broader Impacts

Section 6 (“Broader impact”) gives the high-level positive/negative/scope structure for the main body. We expand here on five lower-level questions that the main paragraph elides for length: foundation-model access asymmetry and reproducibility-equity, the net carbon accounting of trading env steps for VLM calls, the dual-use surface of the generator–verifier pattern, an explicit *What this work is not* scope statement, and the safety properties of training-time-only foundation-model use. The purpose of the longer treatment is to surface considerations that a NeurIPS Broader-Impacts reader can use to evaluate the work without having to derive them from the methodological body.

Reproducibility-equity and foundation-model access asymmetry. The two leading VLMs we evaluate (Claude Opus 4.7 and GPT-4o) are closed-API products available only through paid services from Anthropic and OpenAI respectively. Replicating the full paper-grade sweep costs $\approx \$320$ in inference fees (Appendix D.2); the in-flight verified-CF sweep alone is $\approx \$85$ per run. This places a non-trivial floor on the researcher cohort that can reproduce or extend the method as specified, and is a material reproducibility-equity concern that *open-data* reproducibility checklists alone do not surface. Three structural mitigations are built into the codebase release. (a) *Heuristic-Oracle fallback.* The Oracle v3 heuristic (Appendix B.3) provides a fully VLM-free credit-assignment oracle that uses only state-geometry and the env’s own reward function. It runs on CPU, requires no API key, and reproduces the mechanism (PER prior $\times$ semantic boost) at zero foundation-model cost. The headline Path C kill experiment was decided against Oracle-CF at the 250 k-step horizon, so the load-bearing decision in the paper does not require API access to replicate. (b) *Open-weights VLM extension is pre-registered.* We pre-register an open-weights extension (LLaVA-NeXT, Qwen2.5-VL-7B, Pixtral) in §5.5 as the natural robustness follow-on. These models run on a single 16 GB consumer GPU and require zero per-call API cost; the published commit includes a shim layer in `src/vlm/` so that the same prompt template addresses an open-weights endpoint without code change. (c) *Per-call cost and rate-limit reporting.* The codebase release ships with a `COST.md` that lists per-call token-count and dollar-cost on a representative sweep, so a reader contemplating a replication can compute an a-priori budget without having to run the pipeline. The same file is the basis of the Appendix D.2 dollar figures.

Net carbon accounting: env steps traded for VLM calls. Sample-efficient methods reduce GPU-hours by reducing env-step count, but VLM inference is itself an energy expenditure on a distant data center. The net carbon delta of trading env steps for VLM calls is not a priori positive. For our scheme the accounting is: 1.7×10^3 VLM calls per 500 k-step run (Appendix D.2; one call per failed episode at the `vlm_call_interval` = 16 default), each $\leq \$0.05$ and ~ 8 kJ of inference energy on GPT-4o’s reported tokens-per-Joule profile, vs. a 500 k-step SAC run consuming ~ 3.5 GPU-hours on A10G (≈ 2 MJ at 150 W). VLM inference is therefore $\sim 10^{-2}$ of the training run’s energy at the call rate we use; the sample-efficiency gain dominates. The conclusion would invert if either (i) the call-rate budget were raised by two orders of magnitude (e.g., one VLM call per env-step rather than per failed episode), or (ii) the training run were itself short (≤ 50 k steps), at which point the cross-over point is reached. We document this so future work scaling the call rate has a numeric basis for the trade-off.

Dual-use surface of the generator-verifier pattern. The verified-CF acceptance gate is a general-purpose pattern: a foundation-model generator produces a low-bandwidth structured proposal (here, a 4-vector action), and a deterministic verifier either accepts or rejects based on a substrate test (here, MuJoCo sparse reward firing). The pattern transfers naturally to program-synthesis-with-tests [35], constraint satisfaction with SMT verifiers, and lemma-suggestion in interactive theorem proving. The dual-use surface comes from the same structural strength: a generator that learns to *satisfy the verifier without satisfying the underlying intent* can be a vector for specification gaming. In robotics this is bounded by the env’s sparse-reward predicate (the verifier accepts only physically-correct goal achievement), but in domains where the verifier is itself learned, partial, or game-able (e.g., proof-checking with axiom schemas the prover can introduce, or test-suite coverage that the generator can adversarially target), the same accept/reject design admits a Goodhart’s-law failure mode that this paper’s setting does not exhibit. We flag this because the methodological recipe will travel ahead of its physics-grounded verifier; readers transplanting the pattern to non-physics domains should explicitly audit whether the verifier is closed under the generator’s output distribution or can be elicited to fire on outputs that fail the original objective.

Safety properties of training-time-only foundation-model use. A structural property of our scheme worth surfacing: the foundation model is invoked only at *training time*, only on offline replay data, and only to influence the buffer’s sampling distribution (Semantic PER) or to propose a candidate relabel that is then verified by the env itself (Verified CF). The deployed policy is a standard SAC actor network; it carries no VLM weights, makes no foundation-model API call at inference, and has no dependence on the foundation-model provider’s API uptime or behavior. This compares favorably to recent VLM-as-policy and VLM-as-reward lines [24, 33]: a VLM specification drift, API deprecation, or adversarial-prompt incident after training has no effect on the trained agent’s runtime behavior. The reproducibility implication is also positive: re-running the training pipeline

several months later against a different VLM checkpoint produces a different sampling distribution, but the trained policy is a function of the env’s true sparse reward and the SAC objective, both of which are stable. The implication for deployment audits is that a regulator inspecting the agent’s runtime behavior does not need access to the VLM that was used during training.

What this work is not. We make four explicit non-claims, since each has been a common misreading pattern in adjacent VLM-in-RL work and we want the broader-impacts reader to have an unambiguous scope statement. (N1) This is not a sim-to-real claim. All experiments are simulation-only on three Gymnasium-Robotics Fetch envs. The heuristic and verifier both depend on a forkable, sparse-reward training simulator, which is structurally unavailable in real-robot training. Section 6 bullet (vi) discusses what would be required to extend. (N2) This is not a foundation-model evaluation paper. We do not benchmark VLMs against each other on a fixed VLM eval suite; when a sub-claim does require a model comparison (§5.3, §5.5) we report it on $n \leq 10$ episodes per cell with disjoint Clopper–Pearson intervals, and we are explicit that the load-bearing finding is *prompt-architectural rather than vendor-specific*. (N3) This is not a multi-agent or human-interaction system. The VLM is invoked offline on replay frames; it does not interact with a human user during training, does not generate natural-language output that is shown to a deployer, and does not act as a teacher in a teacher–student RL loop. (N4) This is not an autonomy claim. The agent we train is a single-task goal-conditioned policy with a 50-step horizon; we make no claim about open-ended task selection, multi-step planning, or self-improvement.

Personal data, consent, and content moderation. No human-subjects data, identifiable images, or personal text are collected, generated, or processed at any pipeline stage. The prompt corpus consists of rendered MuJoCo frames of simulated gripper-and-block scenes, which contain no faces, text, license plates, or identifiable scene content. The VLM API providers’ standard content policies apply to our calls, but no policy-violating content was ever generated or attempted; we filed no abuse reports and received none. The codebase release contains no copyrighted images, no scraped web content, and no human-annotated labels.

F.3 Compute infrastructure details

For full reproducibility, the exact environment specifications used:

- **OS:** Ubuntu 24.04 LTS (kernel 6.17), glibc 2.39.
- **Python:** 3.10.14, managed via the project’s `.venv` (pip).
- **PyTorch:** 2.4.1+cu124 on CUDA 12.4, cuDNN 9.1.
- **Gymnasium:** 1.0.0; **Gymnasium-Robotics:** 1.3.0; **MuJoCo:** 3.1.6; **mujoco-py:** not used (pure MuJoCo Python bindings).
- **Rendering:** EGL backend (`MUJOOCO_GL=egl`) for headless Modal containers; OSMesa for local debugging.
- **VLM SDKs:** anthropic 0.39.0, openai 1.55.0; both pinned in `requirements.txt`.
- **Other:** numpy 1.26.4, Pillow 11.0.0, pyyaml 6.0.2, wandb 0.18.7.
- **Modal image:** `python:3.10-slim` base, plus `libegl1-mesa libgl1 libosmesa6 libosmesa6-dev`.

A `Dockerfile` reproducing this exact environment is included in the codebase release under `docker/` (for non-Modal users).

G Failure Mode Analysis

G.1 The teleport-collapse failure mode (C1v2-A)

Formal characterization. The teleport-collapse failure mode of VLM counterfactual prompting is the event $\|p_{cf} - g\| < \rho$ where p_{cf} is the VLM’s emitted `corrective_position` for a failed episode and g is the desired-goal of that episode (with $\rho = 0.05$ m). Empirically, this mode appears in:

- Claude Opus 4.7, FetchPush, `ACHIEVED_GOAL`: 4/4 episodes (100%).

- Claude Opus 4.7, FetchPickAndPlace, ACHIEVED_GOAL: 3/4 episodes on real, 2/2 on synthetic.
- GPT-4o, FetchPush, ACHIEVED_GOAL: 0/6 on synthetic; on the joint ALL variant: 4/6 on synthetic.
- Claude Sonnet 4.5, FetchPush, ACHIEVED_GOAL: 3/4 on real (75%).

Why it happens. The ACHIEVED_GOAL prompt asks for “the position the object should have reached at the failure frame for the episode to be on track”. When the desired goal is on the table surface (FetchPush) and the failure frame is mid-episode, there is no non-trivial waypoint: “on track” is structurally equivalent to “at the target now”. The VLM resolves this ambiguity by output $p_{cf} = g$ exactly. On PickAndPlace, the goal is in mid-air ($z > 0.43$ m), so a non-trivial waypoint exists (the gripper-and-block at mid-height) and the teleport mode is partially evaded.

Mitigations evaluated.

1. **Hard rejection gate.** Reject all outputs with $\|p_{cf} - g\| < \rho$. Effective at gate-time but loses signal ($\sim 100\%$ of FetchPush ACHIEVED_GOAL outputs rejected for Claude Opus 4.7). Adopted as a belt-and-suspenders fallback in the production pipeline.
2. **Prompt-variant switch to ALL.** The unified ALL prompt structurally regularizes the position field (must co-emit a position *and* a 4-D action), reducing teleport rate on real data from 100% to 17–25%. Helpful but not eliminative.
3. **Model switch.** Switching Opus 4.7 to Sonnet 4.5 rescues PickAndPlace (75% $\rightarrow$ 0%) but not Push (75% persists). Switching to GPT-4o eliminates Push on synthetic (100% $\rightarrow$ 0%) but persists on the joint ALL variant.
4. **Mechanism replacement (verified-CF).** Switch to the ACTION variant + simulator-fork verification (Algorithm 2). Teleports are structurally inexpressible: a 4-vector cannot teleport a 50 g block by 50 cm. This is the adopted mechanism.

Adopted production configuration. (GPT-4o, ACHIEVED_GOAL with $\rho = 0.05$ gate, verified-CF fallback). The combination of (i) GPT-4o’s strict dominance on the synthetic ACHIEVED_GOAL grid, (ii) the gate catching residual teleports, and (iii) the verified-CF mechanism absorbing rejections gives a defense-in-depth strategy.

G.2 Action sign-flip in real-data evaluation (C1v2-B)

Phenomenon. On the ACTION prompt variant, $\sim 50\%$ of VLM-emitted 4-vectors have at least one axis whose sign disagrees with $\text{sign}(g - \text{ag}_K)$ for the failure-frame achieved goal. The VLM’s textual explanation *correctly* describes the failure mode in 4/4 episodes (e.g., “descend and close the gripper”), but the emitted action vector points in the wrong axis-direction. The judge catches each contradiction with phrasing like “the action’s negative-y component moves away from the target”.

When it occurs. The sign-flip rate per (source, model, variant):

- Synthetic, Opus 4.7, ACTION: 0.50 ($n=4$).
- Synthetic, Opus 4.7, ALL: 0.17 ($n=2$).
- Real, Opus 4.7, ACTION: 0.55 ($n=10$).
- Real, Sonnet 4.5, ACTION: 0.70 ($n=5$).
- Real, Opus 4.7, ALL: 0.39 ($n=9$).

Hypothesized cause. VLMs have known difficulty grounding metric 3-D axis directions (“which way is +y”) from 2-D rendered keyframes. The Fetch envs use an angled tabletop camera; the y-axis appears as a horizontal in-plane axis with no visual anchor, and the VLM frequently confuses its sign. The ALL variant reduces this rate by forcing the VLM to co-emit a position field (which it tends to get right) alongside the action, providing implicit cross-checking.

Recommended sign-check gate. The recommended mitigation for any pipeline consuming ACTION outputs is the simple gate

$$\text{accept iff } \text{sign}(a_{\text{cf}}[3]) = \text{sign}(g - \text{ag}_K) \quad \text{component-wise on axes with } |g - \text{ag}_K| > 0.01 \text{ m.} \quad (12)$$

Empirically this filters $\approx 50\%$ of ACTION outputs. The remaining $\sim 50\%$ have correct signs and are forwarded to either the verified-CF mechanism or direct action-relabeling.

Why the verified-CF mechanism subsumes this. The simulator-fork verification (Algorithm 2) is *strictly stronger* than the sign-check gate: an a_{cf} with a flipped sign on any axis points the gripper away from the goal, so the verifier env rolls out for 50 steps without firing the sparse reward, and the counterfactual is rejected as `goal_not_reached`. The sign-check gate is therefore mostly a diagnostic and a pre-filter to save the verifier’s 17.6 ms per call; on the production pipeline the verifier alone suffices.

G.3 VLM API rate-limit at high parallelism (C1v2-C)

Phenomenon. When running 18 parallel SAC+HER+Verified-CF seeds on Modal A10G instances with `vlm_call_interval=1` (every failed episode) and model `claude-sonnet-4-5`, the Verified-CF mechanism silently degrades: 73% of VLM calls return `None` after retry exhaustion rather than a usable counterfactual. Modal logs confirm the cause is HTTP 429 errors from Anthropic’s Tier 1 rate limits (5 req/min and 10,000 tokens/min for `claude-sonnet-4-5`). The burst arrival rate of 18 seeds $\times \sim 1$ call/episode-end easily saturates both ceilings; the retry logic exhausts its budget and increments the `buffer/cf_vlm_returned_none` counter rather than raising an exception, so the degradation is invisible without explicit monitoring.

Why this is structural, not method-specific. Any VLM-in-replay scheme in which VLM calls are triggered *per failed episode* has a call rate that scales linearly with the number of parallel training runs: $\lambda_{\text{VLM}} \propto P \cdot \rho_{\text{fail}}$ where P is run parallelism and ρ_{fail} is the per-step failure rate. For `FetchPickAndPlace` during the first 200,000 steps, $\rho_{\text{fail}} \approx 0.95$, so even a single run at `vlm_call_interval=1` makes $\sim 0.3\text{--}0.5$ calls/sec wall-clock. At $P = 18$ this is $\sim 5\text{--}9$ calls/sec, an order of magnitude above the Tier 1 RPM ceiling.

Mitigations for practitioners. Three strategies are effective in isolation; in practice a combination is recommended:

1. **Upgrade API tier.** Anthropic Tier 2+ raises RPM to ≥ 50 req/min, sufficient for $P \leq 6$ at `vlm_call_interval=1`. This is the simplest fix for large sweeps.
2. **Dilute call rate via `cf_call_interval`.** Setting `cf_call_interval = k` calls the VLM only on every k -th failed episode, reducing λ_{VLM} by factor k . For $k = 8$ and $P = 18$ the burst rate drops to $\sim 0.06\text{--}0.11$ calls/sec, well within Tier 1 limits. The trade-off is reduced counterfactual data density; empirically $k = 4\text{--}8$ retains acceptable relabeling frequency.
3. **Split VLM providers across runs.** Routing half the seeds to GPT-4o (OpenAI) and half to a Claude model distributes the call load across independent rate-limit pools. For our production pipeline, GPT-4o at `vlm_call_interval=1` remains within OpenAI Tier 1 limits up to $P \approx 12$.

A batch-API approach (collecting failed episodes across a full epoch, then submitting a single bulk request) is a fourth option for offline or non-latency-critical replay augmentation, but it is incompatible with the online episode-triggered call pattern used here.

Implications for reported experiments. Phase 2 attempts 3 and 4 of our in-flight Verified-CF sweep were stopped after health checks revealed 73% none-rates. Affected runs are excluded from all quantitative comparisons. The Phase 1 ablation (Section 5.2) uses `cf_call_interval=8` throughout and was not affected; the Semantic-PER results reported there use GPT-4o as provider and were collected before the `claude-sonnet-4-5` switch, so no runs in Figure 6 are contaminated.

H Reproducibility commands

For users wishing to reproduce specific results:

Headline ablation (Figure 6).

```
# Local single-seed smoke
python train.py --config configs/her_semantic_per.yaml \
  --env.name FetchPickAndPlace-v4 --training.seed 42 \
  --training.total_steps 50000

# Full Modal sweep (27 runs)
modal run modal_app.py::run_ablation
```

Counterfactual prompt evaluation (Tables 3).

```
# Synthetic (6 episodes)
python scripts/test_counterfactual.py \
  --provider openai --model gpt-4o \
  --judge_model claude-opus-4-7 \
  --variants narrative action achieved_goal all \
  --n_episodes 6 --budget 60

# Real (10 episodes)
python scripts/run_clv2_real_data_eval.py \
  --n_episodes 10 --providers openai anthropic
```

Cross-task transfer (Figure 8).

```
python scripts/n2_validate_cross_task.py \
  --envs FetchPush-v4 FetchPickAndPlace-v4 FetchSlide-v4 \
  --n_per_env 4 --K 5
```

Verified-CF smoke (Figure 4).

```
python scripts/n1_smoke_verified_cf.py
# 4/4 expected pass; results in agent_reports/N1_smoke_results.json
```

I Counterfactual Prompt Design: Extended Analysis

This appendix expands the §5.3 prompt-design ablation along four axes the main body refers to but does not develop in detail.

Mechanism of the position-copy attractor. §4.2 gives the mechanistic account at the output-schema level; the empirical signature is that the four ACHIEVED_GOAL Opus calls matched `corrective_position` to `desired_goal` within ≤ 1 cm in 4/4 cases; on ALL the match held in 2/2 Opus calls. GPT-4o’s 0/6 on ACHIEVED_GOAL vs. 4/6 on ALL is the discriminating signature: the only structural difference between the two prompts is the inclusion of the full action-history block in ALL, which lengthens the prompt and crowds out the late-prompt discouragement of position-copying. This is consistent with prompt-length-dependent attentional dilution: the in-context goal triplet remains the strongest local numerical attractor while the discouraging instruction is pushed out. The interpretation is testable: shortening the ALL variant’s action history should monotonically reduce GPT-4o’s teleport-collapse rate. We flag this as a falsifiable prediction for the camera-ready follow-on.

Judge-bias and inter-rater reliability. The judge for Table 3 is Claude Opus 4.7, which is also one of the two candidate models. This same-vendor judge-candidate overlap is the most direct external-validity threat to the table: the 0.97 goal-progress score that Opus’s degenerate goal-copy receives is exactly the failure mode a self-preferring judge would amplify, since the prediction matches the rubric’s “goal-progress” anchor mechanically. We mitigate in three ways. (i) The teleport-collapse metric is computed by predicate $\mathbb{K}[\|\hat{p}-g\|_2 \leq d_{th}]$, so the load-bearing 0/6 vs. 4/4 comparison is judge-independent. (ii) Plausibility is computed against a fixed 5-point rubric in the system prompt (Appendix A), which constrains the response space relative to free-form scoring. (iii) The §5.5 replication on real data uses Sonnet 4.5 as the candidate, removing the same-vendor judge-candidate edge case for the load-bearing GPT-4o comparison. If Opus-as-judge over-weights

the Opus NARRATIVE 0.79 relative to ground truth, the table’s effect would be to inflate Opus’s plausibility rank, biasing *against* the GPT-4o adoption claim—the safer of the two mis-specifications.

Statistical power and pre-registered falsification. Cell sizes are $n=6$ on the GPT-4o rows and $n=2-4$ on the Opus rows (Opus calls re-used from C1’s partial grid to avoid re-billing). At $n=6$ a 0/6 outcome is consistent with any true teleport-collapse rate ≤ 0.39 at $\alpha=0.05$ (Clopper–Pearson upper); at $n=4$ a 4/4 outcome is consistent with any true rate ≥ 0.40 . The two intervals are disjoint, so the 0/6 vs. 4/4 contrast between (GPT-4o, achieved_goal) and (Opus, achieved_goal) is the strongest single binary-classification claim the table makes. The plausibility/goal-progress numbers are weaker, with $n=4$ SEs of 0.04–0.18 that admit ranking swaps under any single retry. We pre-register the following falsification: a 30-episode re-evaluation of (GPT-4o, achieved_goal) on FetchPickAndPlace with the production prompt should show a teleport-collapse rate $\leq 5\%$. A higher rate falsifies the prompt-architectural claim and reverts the recommendation to ACTION alone.

IS-posterior precondition framing. The prompt-design ablation sits upstream of the §3 posterior framing: q_ϕ is measured here in the specific case where the localization is of a *corrective* timestep, not a *failure-causing* one. A prompt that induces position-copying yields a degenerate q_ϕ that places posterior mass on desired_goal regardless of τ , breaking the trajectory-conditional structure that makes the IS-correction analysis of Eq. (4) go through. The (GPT-4o, achieved_goal) configuration is thus the minimal prompt-design constraint that keeps $q_\phi(\cdot | \tau)$ τ -dependent at all. Read this way, the prompt-design experiment is not a stand-alone engineering ablation but a *precondition* for the theoretical contribution: without it the multiplicative-form story of §3 would be specified over a posterior that is in fact τ -independent for two of the four output schemas we evaluate.

Output-type-to- q_ϕ degeneracy map. The teleport-collapse predicate $\|\hat{p}-g\|_2 \leq d_{th}$ is the empirical signature of a trajectory-independent posterior $q_\phi(\hat{p} | \tau) = \delta(\hat{p}-g)$, and the two-regime taxonomy of Appendix C.3 (“Two regimes of q_ϕ -degeneracy”) fixes the operational meaning of that Dirac on the verifier channel of §4.3. The four-by-two map is: (NARRATIVE, any model) $\Rightarrow m_{gen}=0$ on the verifier channel (the schema does not produce an executable 4-vector); (ACHIEVED_GOAL or ALL, position-emitting model) $\Rightarrow \text{Dirac } q_\phi, m_{ver} \rightarrow 0$; (ACTION, any model) $\Rightarrow$ non-degenerate q_ϕ (numerical attractor absent by construction). The production (GPT-4o, achieved_goal) configuration and the verified-CF mechanism therefore occupy different cells of this map—the former with the rejection gate retained as defense in depth, the latter relying on ACTION for structural immunity.

J Cross-Task Transfer: Per-Episode Details

The cross-task transfer pilot of §5.4 tested 12 failed episodes (4 per Fetch env) with the single format-string prompt of Appendix A differing only in the task-description sentence. The judge-rated metrics (parse rate, task-relevance, plausibility) are 12/12 on every axis. The tolerant keyframe agreement gradient—Slide (4/4) > Push (3/4) > PickAndPlace (2/4)—is attributable to heuristic Oracle under-localization rather than VLM error: on Slide the ballistic heuristic and the keyframe images both identify the puck launch event; on Push the contact-loss event is visually salient; on PickAndPlace the “grip failed” failure spans $\sim 1-3$ consecutive timesteps with little frame-to-frame visual change, so the heuristic argmin localizes to closest-approach while the VLM assigns the failure one step earlier at the decisive grip sequence—a different but arguably more physically-correct localization. We pre-register the $n \geq 30$ -per-task sweep and non-Fetch transfer (Adroit, OGBench, MiniGrid for a head-to-head with VLM-RB) as the quantitative follow-on.

K Real-Data Validation: Extended Caveats

Episode collection. The 10 real failures of §5.5 were drawn from two SAC+HER checkpoints trained from scratch under the §5.1 hyperparameters: FetchPickAndPlace-v4 for 50k steps and FetchPush-v4 for 30k steps, both deliberately stopped at $\sim 5\%$ deterministic eval success—well below the $\sim 500k/\sim 80k$ convergence horizons. Stochastic 50-step rollouts (actor mean plus tanh-squashed Gaussian tail) yielded 10 failures (6 PickAndPlace from seeds 50000–50102, 4 Push from seeds 60000–60051; data-collection success rate $\approx 7\%$). The heuristic localizer’s mid-clip safety rule did not engage on any of the 10 real episodes.

Two-vendor reframe. The synthetic sweep paired Claude Opus 4.7 against OpenAI GPT-4o. For the real-data sweep the OpenAI key returned a hard `insufficient_quota` block, so we substituted Claude Sonnet 4.5 as the second generator. Empirically Sonnet behaves closer to GPT-4o on PickAndPlace (0/6 vs. Opus’s 3/4) and closer to Opus on Push (3/4 vs. Opus’s 4/4): the intra-vendor variation spans most of the cross-vendor range. The appropriate strength of claim is therefore that teleport-collapse is prompt-architectural and task-conditioned, not vendor-/family-specific. We pre-register an open-weights VLM extension (LLaVA-NeXT or Qwen2.5-VL-7B [2]) as the natural next robustness check.

Caveats. (i) $n = 10$ episodes is a pilot, not a powered sweep; we pre-register $n \geq 30$ per environment with bootstrapped CIs. (ii) The OpenAI quota block prevented a within-pipeline GPT-4o real-data comparison; the Sonnet 4.5 substitution is informative for the intra-vendor question but does not replace GPT-4o on real data. (iii) The judge (Opus 4.7) overlaps with one of the generators on the synthetic side; the Sonnet-vs-Opus contrast is internally valid because the *same* judge scores both generators. A cross-judge sanity check (Sonnet 4.5 or GPT-4o judging the same outputs) is on the open-work list.

L Additional Limitations

The main body (§6) discusses four limitations; this appendix records five additional ones that the body refers to.

(v) Cross-task evidence beyond Fetch. Our cross-task evidence is at $n = 12$ in three Fetch envs; transfer to non-Fetch tasks (Adroit, OGBench, robosuite—and to VLM-RB’s MiniGrid benchmark for a head-to-head comparison) is the natural follow-on.

(vi) VLM-family robustness. Our robustness sweep covers three models from two vendors (Opus 4.7 and Sonnet 4.5 from Anthropic; GPT-4o from OpenAI); in terms of pre-training lineage it is effectively a two-family comparison, since Opus and Sonnet share substantial common corpus and RLHF pipeline. Open-weights VLMs (LLaVA-NeXT, Qwen2.5-VL, Pixtral) are not evaluated; the relative magnitude of the prompt-architectural teleport failure on smaller open VLMs is an open question.

(vii) Simulator-fork requirement. The simulator-as-verifier assumption requires a forkable training environment; tasks with non-deterministic or real-robot dynamics would need an additional verification layer.

(viii) Single-timestep vs. clip granularity. Single-timestep localization vs. clip-level aggregation is an untested trade-off. VLM-RB’s 32-frame sub-trajectory signal averages over more visual context and may be more robust to single-frame VLM attention errors; we adopt the single-timestep form because it is a prerequisite for the counterfactual relabeling step (clip-level signals cannot identify a snapshot from which to fork the simulator). Whether the localization granularity independently improves credit assignment remains unablated.

(ix) Architectural-vs.-empirical differentiation from VLM-RB. The methodological differentiation from VLM-RB [27] in Table 1 rests on five axes, but the comparison is *architectural*, not empirical: we do not run Semantic PER on MiniGrid or OGBench, nor VLM-RB on FetchPickAndPlace. Our Sharony reproduction in the codebase (§5.6 (D)) is pre-staged for head-to-head evaluation on Fetch but the comparison sweep has not yet launched.